

High performance E-Commerce

إعداد :

- نغم خلدون مجاهد
- رغد مازن الخوص
- زياد راتب سليمان
- زين علاء الدين رحمون
- محمد نور شيخموس شيخموس

رابط الغيت :

<https://github.com/NaghamMujahed/High-performance-E-Commerce>

بإشراف المهندس حذيفة

الطلب الاول :

1.المشكلة

عند وصول عدة مستخدمين بنفس الوقت لتعديل كمية منتج في المخزون، يحدث تضارب في البيانات (Race condition)

مثال:

- كمية منتج معين 1
- مستخدم A قرأ الكمية يلي بتساوي 1
- مستخدم B قرأ الكمية يلي بتساوي 1
- مستخدم A اشترى $\leq$ الكمية صارت 0
- مستخدم B اشترى كمان $\leq$ الكمية صارت -1

النتيجة :بيانات خاطئة وخسارة في المخزون.

2.الحلول المقترحة

الحل الأول (بدون حماية)

- كل مستخدم يقرأ ويعدل بحرية
- لا يوجد أي تحكم في الوصول المتزامن
- النتيجة :تضارب وبيانات خاطئة

الحل الثاني Optimistic Locking

- يسمح لجميع المستخدمين بالقراءة بحرية
- يضيف حقن Version لكل سجل
- عند الحفظ يتحقق إذا تغيرت البيانات
- إذا تغيرت يرفع خطأ OptimisticLockException
- مناسب للأنظمة التي تكون فيها القراءة أكثر من الكتابة

الحل الثالث ال synchronized

- يضمن أن thread واحد فقط يستطيع تنفيذ الكود المحمي في نفس الوقت بإضافة كلمة synchronize
- سبب عدم اختياره لأنه يعمل على مستوى الكود فقط
- يعني إذا كان النظام يعمل على أكثر من سيرفر فكل سيرفر سيكون له قفله الخاص ولا يعلم بالسيرفرات الأخرى مما يجعله غير كاف لحماية البيانات

الحل الرابع (المختار) Pessimistic Locking

- يقفل السجل عند القراءة
- لا يسمح لأي مستخدم آخر بالقراءة أو التعديل حتى ينتهي الأول
- يضمن عدم حدوث أي تضارب في البيانات
- مناسب للبيانات الحساسة مثل المخزون
- يحمي البيانات من التضارب على مستوى قاعدة البيانات DB

3. سبب اختيار Pessimistic Locking

تم اختيار Pessimistic Locking لعدة أسباب:

- المخزون بيانات حساسة جداً — لا يمكن قبول أي خطأ
- لا يمكن السماح بقراءة بيانات قديمة (Dirty Read)
- Spring Data JPA يدعمه مباشرة بـ @Lock
- يضمن سلامة البيانات تحت الضغط العالي

4. نتائج الاختبار بـ Jmeter و ال AOP

تم اختبار النظام بـ 100 مستخدم متزامن على نفس المنتج:

قبل التحسين

صورة 1 :

من التيرمنال باستخدام مفهوم ال (LOG) AOP

نوضح في هذا الصورة كل ثريد متى انتهى وما هو الوقت الذي استغرقة وانتهى فيه

```
2026-05-15T13:54:14.293+03:00 INFO 15724 --- [demo] [o-8080-exec-100] c.example.demo.aspect.PerformanceAspect : [PURCHASE] Started: purchaseWithoutLock | Thread: http-nio-8080-exec-100
2026-05-15T13:54:14.318+03:00 INFO 15724 --- [demo] [io-8080-exec-24] c.example.demo.aspect.PerformanceAspect : [PURCHASE] Finished: purchaseWithoutLock | Time: 1190ms | Thread: http-nio-8080-exec-24
2026-05-15T13:54:14.334+03:00 INFO 15724 --- [demo] [nio-8080-exec-9] c.example.demo.aspect.PerformanceAspect : [PURCHASE] Finished: purchaseWithoutLock | Time: 1224ms | Thread: http-nio-8080-exec-9
2026-05-15T13:54:14.494+03:00 INFO 15724 --- [demo] [nio-8080-exec-9] c.example.demo.aspect.PerformanceAspect : Finished: ProductService.purchaseWithoutLock | Time: 1421ms
2026-05-15T13:54:14.348+03:00 INFO 15724 --- [demo] [nio-8080-exec-4] c.example.demo.aspect.PerformanceAspect : Finished: ProductService.purchaseWithoutLock | Time: 1263ms
2026-05-15T13:54:14.375+03:00 INFO 15724 --- [demo] [nio-8080-exec-2] c.example.demo.aspect.PerformanceAspect : [PURCHASE] Finished: purchaseWithoutLock | Time: 1261ms | Thread: http-nio-8080-exec-2
2026-05-15T13:54:14.497+03:00 INFO 15724 --- [demo] [nio-8080-exec-2] c.example.demo.aspect.PerformanceAspect : Finished: ProductService.purchaseWithoutLock | Time: 1421ms
2026-05-15T13:54:14.458+03:00 INFO 15724 --- [demo] [io-8080-exec-21] c.example.demo.aspect.PerformanceAspect : [PURCHASE] Finished: purchaseWithoutLock | Time: 1347ms | Thread: http-nio-8080-exec-21
2026-05-15T13:54:14.501+03:00 INFO 15724 --- [demo] [io-8080-exec-21] c.example.demo.aspect.PerformanceAspect : Finished: ProductService.purchaseWithoutLock | Time: 1426ms
2026-05-15T13:54:14.489+03:00 INFO 15724 --- [demo] [io-8080-exec-24] c.example.demo.aspect.PerformanceAspect : Finished: ProductService.purchaseWithoutLock | Time: 1406ms
2026-05-15T13:54:14.492+03:00 INFO 15724 --- [demo] [io-8080-exec-20] c.example.demo.aspect.PerformanceAspect : [PURCHASE] Finished: purchaseWithoutLock | Time: 1368ms | Thread: http-nio-8080-exec-20
2026-05-15T13:54:14.549+03:00 INFO 15724 --- [demo] [io-8080-exec-20] c.example.demo.aspect.PerformanceAspect : Finished: ProductService.purchaseWithoutLock | Time: 1467ms
```

من الصورة نستخرج :

exec-24: Finished | Time: 1190ms

exec-9: Finished | Time: 1224ms

exec-4: Finished | Time: 1263ms

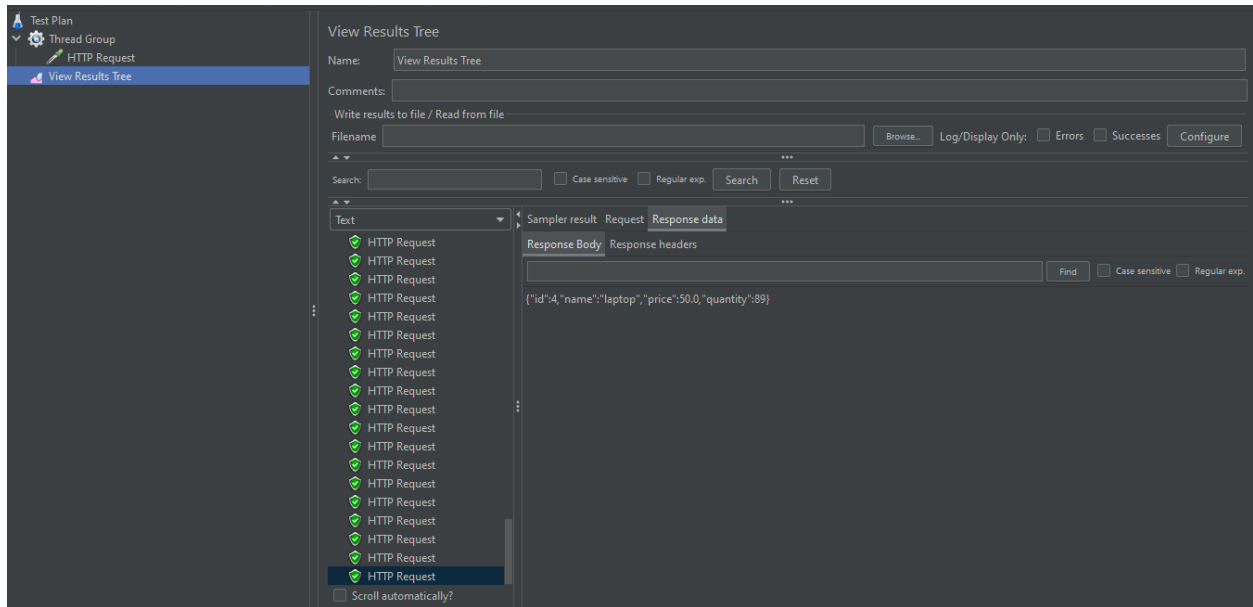
exec-2: Finished | Time: 1261ms

يعني كل ال Threads خلصو بنفس الوقت تقريبا هاد دليل انهم اشتغلوا بالتوازي بدون انتظار .

صورة 2 :

صورة من ال Jmeter

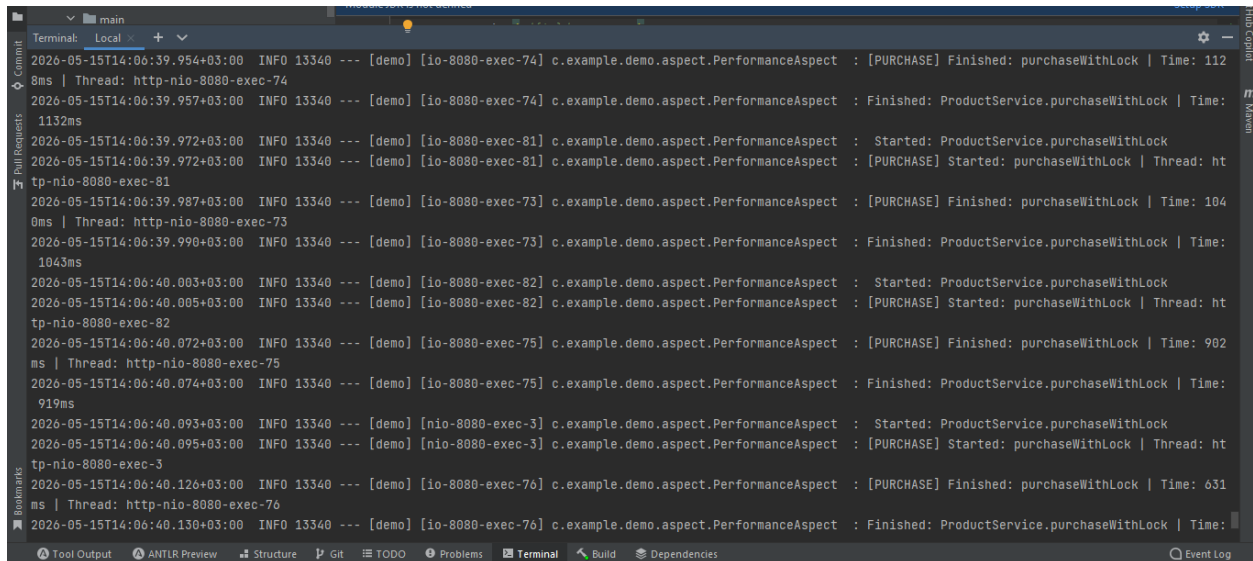
المنتج له 100 كمية اصبحت في اخر طلب 89 وهذا الشيء خاطئ .



بعد التحسين :

صورة 1 :

باستخدام مفهوم الـ AOP(LOG)



من الصورة نستخرج :

exec-74: Finished | Time: 1128ms

exec-73: Finished | Time: 1040ms

exec-75: Finished | Time: 902ms

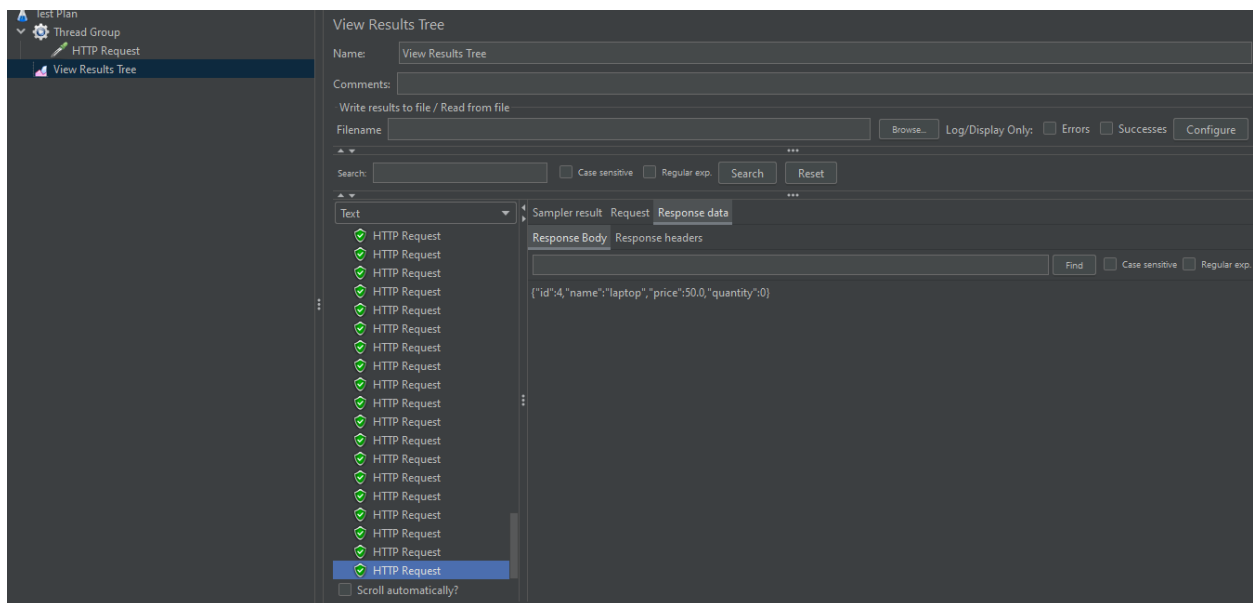
exec-76: Finished | Time: 631ms

نلاحظ ان لكل thread وقت مختلف يعني ان كل thread انتظر ال lock يتحرر قبل ما يشتغل مو كلهم بنفس الوقت .

صورة 2 :

صورة من ال Jmeter

بعد التحسين وازافة ال thread safety اصبحت الكمية 0 وهذا الحل الصحيح



الطلب الثاني :

المشكلة الأساسية :

في الأنظمة التي تستقبل عدد كبير من الطلبات المترامنة، لا تكمن المشكلة فقط في تنفيذ العمليات بالتوازي، وإنما في كيفية إدارة الموارد الحاسوبية بكفاءة دون استنزاف النظام.

في مشروعنا كانت عملية الشراء (Purchase Operation) تتضمن عدّة خطوات :

- قراءة المنتج من قاعدة البيانات
- قفل المنتج لحماية البيانات
- تحديث الكمية
- حفظ التعديلات (أي حفظ البيانات)
- انتظار زمني (Processing Time) لمحاكاة العمليات الواقعية وقد طبقنا هذا التأخير من خلال استخدام :

○ Thread.sleep(1500)

وهذا يعني أن كل عملية شراء تبقى قيد التنفيذ لفترة زمنية قبل أن تنتهي .

لماذا كانت هذه مشكلة :

لأن نموذج One Thread per Task غير قابل للتوسّع عند وجود عدد كبير من الطلبات المترامنة وذلك لأن كل طلب يحتاج إنشاء Thread مستقل .

ومع ارتفاع عدد المستخدمين المترامين يبدأ النظام بإنشاء عدد كبير من الـ Threads وهذا يؤدي إلى:

- استهلاك كبير للذاكرة
- ضغط مرتفع على CPU
- زيادة Context Switching
- انخفاض الأداء
- بطء الاستجابة
- قابلية التوسّع محدودة

ومع الضغط العالي قد يصل النظام إلى: Thread Exhaustion أي عدم القدرة على إنشاء Threads إضافية

الهدف من هذا الطلب هو التحكم بعدد العمليات المتوازية وتحسين إدارة الموارد الحاسوبية بحيث يستطيع النظام :

- تحمل عدد كبير من المستخدمين
- منع استنزاف الموارد
- الحفاظ على استقرار الأداء
- زيادة قابلية التوسع
- تحسين استجابة النظام تحت الضغط العالي

الحلول المقترحة :

قبل اختيار الحل النهائي قمنا بدراسة عدّة طرق مستخدمة لإدارة الـ Concurrency والموارد.

أولاً : Traditional Threads

الحل الأبسط كان الاعتماد على الـ Threads التقليدية

- أي أن كل Request يتم تنفيذه داخل Thread مستقل (أي إنشاء Thread جديد لكل Request)

لماذا لم نستخدم Traditional Threads ؟

لأن إنشاء عدد كبير من الـ Threads يؤدي إلى مشاكل كبيرة التي ذكرناها سابقاً (غير مناسب للتعامل مع عدد كبير من المستخدمين)
كما أن كل Thread يعتبر: Heavyweight Thread أي أنها:

- مرتبطة مباشرة بنظام التشغيل
 - تستهلك Memory كبيرة نسبياً
 - إنشاؤها مكلف
 - تسبب Context Switching مرتفع
- ومع زيادة عدد الطلبات يبدأ الأداء بالانخفاض بشكل واضح

ثانياً: Rate Limiting

وهو أسلوب يُستخدم لتحديد عدد الطلبات المسموح بها خلال فترة زمنية معينة. فكرته الأساسية:

- منع المستخدم من إرسال Requests كثيرة جداً
- تقليل الضغط على النظام
- حماية السيرفر من الـ Spam أو الـ Abuse

مثال:

- كل مستخدم يُسمح له بـ 100 Request بالدقيقة
- وأي طلبات إضافية يتم:
- رفضها
- أو تأخيرها

لماذا لم يكن مناسباً كمسألة أساسية عندنا؟

لأن المشكلة التي كنا نحاول حلها لم تكن: تقليل عدد الطلبات القادمة للنظام بل كانت:

كيفية إدارة وتنفيذ عدد كبير من العمليات المتزامنة بكفاءة يعني نحن أردنا أن:

- نستوعب عدد كبير من الطلبات
- وليس منعها

ثالثاً: Thread Pool

والذي يعتمد على إعادة استخدام مجموعة محددة من الـ Threads بدل إنشاء Thread جديد لكل Request

أي الفكرة الأساسية من هذه الطريقة هي :

- إنشاء عدد ثابت من الـ Worker Threads
 - إعادة استخدامهم لتنفيذ عدة Tasks
 - وضع الطلبات الزائدة داخل Queue
- وبالتالي يمنع النظام من إنشاء عدد غير محدود من الـ Threads
لماذا يعتبر Thread Pool حلاً جيداً؟

لأنه:

- يقلل تكلفة إنشاء Threads
- يمنع استنزاف الموارد
- يحسن الاستقرار
- يجعل استهلاك النظام واقعي

حيث أن هذا الحل يُعتبر Safety Valve كما تحدثنا في المحاضرة

لماذا لم نعلم Thread Pool كحل نهائي؟

رغم أن Thread Pool يحسن إدارة الموارد بشكل واضح، إلا أن هناك مشكلة بقيت موجودة وهي أن ال Thread Pool ما يزال يعتمد على: Platform Threads أي Threads حقيقية من نظام التشغيل. وهذا يعني أن:

- كل Thread ما يزال يستهلك Memory
- عدد ال Threads يبقى محدود
- عمليات ال Blocking تبقى ال Thread محجوز وفي مشرونا توجد عمليات انتظار كثيرة مثل:
- Database Access
- Locks
- Thread.sleep

أي أن أغلب الوقت تكون ال Threads بحالة Waiting وليس Processing فعلي وبالتالي بقيت المشكلة الأساسية موجودة عند الضغط العالي.

كما أن زيادة حجم ال Pool بشكل كبير قد تؤدي إلى:

- استهلاك موارد إضافية
- ضغط أعلى على النظام
- زيادة ال Context Switching

لذلك اعتبرنا أن Thread Pool يحسن الأداء جزئياً لكنه لا يقدم حلاً جذرياً.

قرارنا النهائي:

بعد دراسة الحلول السابقة قررنا استخدام: **Virtual Threads**

وذلك لأنها تقدم طريقة أحدث وأكثر كفاءة لإدارة العمليات المتزامنة

سبب الاختيار:

لأن المشكلة الأساسية في مشرونا كانت: تشغيل عدد كبير من العمليات المتزامنة دون استنزاف الموارد الحاسوبية وبناءً على هذه المشكلة ظهر لدينا هذا الحل

كيف تعمل Virtual Threads ؟

بدل إنشاء OS Thread حقيقي لكل Request :

- يتم إنشاء Virtual Thread خفيف جداً
- تقوم JVM بإدارته داخلياً
- يتم تشغيل آلاف Virtual Threads فوق عدد صغير من Carrier Threads

لماذا هذا الحل مناسب جداً لمشرونا؟

لأن مشرونا يحتوي على عمليات كثيرة من نوع: I/O Bound Operations
مثلاً:

- الوصول لقاعدة البيانات
- الانتظار
- الـ Locks
- عمليات الـ Sleep

وكما تحدثنا في المحاضرة أن العمليات الـ I/O Bound تحتاج عدد Threads أكبر من عدد أنوية المعالج بسبب فترات الانتظار الطويلة .
لكن باستخدام Virtual Threads أصبح بالإمكان تشغيل عدد ضخم من العمليات بدون الحاجة لإنشاء عدد ضخم من OS Threads
ماذا يحدث أثناء الانتظار؟

■ في الـ Traditional Threads :

يبقى الـ Thread الحقيقي محجوزاً بالكامل أثناء الانتظار

■ أما في الـ Virtual Threads :

تقوم JVM بتحرير الـ Carrier Thread مؤقتاً لخدمة عمليات أخرى وهذا يؤدي إلى:

- تحسين استغلال CPU
- تقليل استهلاك Memory
- تقليل عدد الـ Threads الحقيقية
- رفع قدرة النظام على تحمل الضغط

كيف قمنا بتطبيق الحل داخل المشروع؟

قمنا بإنشاء مسارين للتنفيذ

الوظيفة	العملية
تنفيذ تقليدي	purchaseWithoutVirtual
تنفيذ باستخدام Virtual Threads	purchaseWithVirtual

لقد قمنا باستخدام @Async ولكن لماذا ؟

للسماح بتنفيذ العمليات بشكل غير متزامن وهذا أدى إلى:

- تنفيذ عدّة عمليات بالتوازي
- عدم حجز الـ Main Request Thread
- تحسين استجابة النظام

أيضاً استخدمنا CompletableFuture :

لإدارة النتائج الناتجة عن العمليات غير المتزامنة وهذا سمح للنظام بـ:

- Async Processing
- تحسين الـ Throughput
- تقليل الـ Blocking
- إدارة أفضل للطلبات المتزامنة

كيف حافظنا على سلامة البيانات؟

رغم تحسين إدارة الموارد بقيت مشكلة: Race Condition موجودة عند محاولة عدة مستخدمين شراء نفس المنتج.

لذلك استخدمنا: Pessimistic Locking التي استخدمناها لحل الطلب الأول وذلك لضمان:

- حماية البيانات
- منع التضارب
- الحفاظ على Data Consistency

ماذا حققنا ؟

- إدارة أفضل للموارد الحاسوبية
- تقليل استهلاك الـ Memory
- تحسين استغلال الـ CPU
- تشغيل عدد كبير من العمليات المتزامنة
- منع استنزاف النظام تحت الضغط العالي
- تحسين الـ Scalability
- تحسين استقرار النظام
- دعم العمليات الـ I/O Bound بكفاءة أعلى

لماذا اعتبرنا هذا القرار الهندسي مناسب أكثر من باقي الحلول؟

لأن الحل جمع بين:

- إدارة الـ Concurrency
- التحكم بالموارد
- تحسين الأداء
- دعم العمليات الـ Blocking
- الحفاظ على سلامة البيانات

وبالتالي حصلنا على نظام:

- أكثر كفاءة
- أكثر استقرارا
- أكثر قابلية للتوسع
- أقرب للأنظمة الحديثة الواقعية

الاختبار قبل وبعد التحسين:

قبل التحسين:

```
File Edit View Navigate Code Refactor Build Run Tools Git Window Help ecommerce_backend1 - with_virtual.jmx
ecommerce_backend1 / src / main / java / com / example / demo / jmeter_files / with_virtual.jmx
Project
Terminal: Local (3)
load-balancer | 2026-05-18T19:20:59.908Z INFO 1 --- [demo] [cat-handler-109] c.example.demo.aspect.PerformanceAspect : [PURCHASE] Finished: purchaseWithoutVirtual | Time: 15087ms |
Thread: tomcat-handler-109
load-balancer | 2026-05-18T19:20:59.901Z INFO 1 --- [demo] [cat-handler-109] c.example.demo.aspect.PerformanceAspect : Finished: ProductService.purchaseWithoutVirtual | Time: 15088ms |
load-balancer | 2026-05-18T19:21:01.409Z INFO 1 --- [demo] [cat-handler-110] c.example.demo.aspect.PerformanceAspect : [PURCHASE] Finished: purchaseWithoutVirtual | Time: 15085ms |
Thread: tomcat-handler-110
load-balancer | 2026-05-18T19:21:01.409Z INFO 1 --- [demo] [cat-handler-110] c.example.demo.aspect.PerformanceAspect : Finished: ProductService.purchaseWithoutVirtual | Time: 15085ms |
load-balancer | 2026-05-18T19:21:02.918Z INFO 1 --- [demo] [cat-handler-111] c.example.demo.aspect.PerformanceAspect : [PURCHASE] Finished: purchaseWithoutVirtual | Time: 15082ms |
Thread: tomcat-handler-111
load-balancer | 2026-05-18T19:21:02.919Z INFO 1 --- [demo] [cat-handler-111] c.example.demo.aspect.PerformanceAspect : Finished: ProductService.purchaseWithoutVirtual | Time: 15083ms |
load-balancer | 2026-05-18T19:21:04.429Z INFO 1 --- [demo] [cat-handler-112] c.example.demo.aspect.PerformanceAspect : [PURCHASE] Finished: purchaseWithoutVirtual | Time: 15083ms |
Thread: tomcat-handler-112
load-balancer | 2026-05-18T19:21:04.429Z INFO 1 --- [demo] [cat-handler-112] c.example.demo.aspect.PerformanceAspect : Finished: ProductService.purchaseWithoutVirtual | Time: 15083ms |
load-balancer | 2026-05-18T19:21:05.943Z INFO 1 --- [demo] [cat-handler-113] c.example.demo.aspect.PerformanceAspect : [PURCHASE] Finished: purchaseWithoutVirtual | Time: 15089ms |
Thread: tomcat-handler-113
load-balancer | 2026-05-18T19:21:05.943Z INFO 1 --- [demo] [cat-handler-113] c.example.demo.aspect.PerformanceAspect : Finished: ProductService.purchaseWithoutVirtual | Time: 15089ms |
load-balancer | 2026-05-18T19:21:07.452Z INFO 1 --- [demo] [cat-handler-114] c.example.demo.aspect.PerformanceAspect : [PURCHASE] Finished: purchaseWithoutVirtual | Time: 15087ms |
Thread: tomcat-handler-114
load-balancer | 2026-05-18T19:21:07.452Z INFO 1 --- [demo] [cat-handler-114] c.example.demo.aspect.PerformanceAspect : Finished: ProductService.purchaseWithoutVirtual | Time: 15087ms |
load-balancer | 2026-05-18T19:21:08.960Z INFO 1 --- [demo] [cat-handler-115] c.example.demo.aspect.PerformanceAspect : [PURCHASE] Finished: purchaseWithoutVirtual | Time: 15082ms |
Thread: tomcat-handler-115
load-balancer | 2026-05-18T19:21:08.960Z INFO 1 --- [demo] [cat-handler-115] c.example.demo.aspect.PerformanceAspect : Finished: ProductService.purchaseWithoutVirtual | Time: 15082ms |
load-balancer | 2026-05-18T19:21:10.472Z INFO 1 --- [demo] [cat-handler-116] c.example.demo.aspect.PerformanceAspect : [PURCHASE] Finished: purchaseWithoutVirtual | Time: 15086ms |
Tool Output JMeter Preview Git TODO Problems Terminal Build Dependencies
Download new built shared indexes. Reduce the indexes time and CPU load with our built JMX and Maven library shared indexes. // Always download // Download once // Don't show again // Confirm (today 0:13 PM) 1/1 1E 11T 8 4 error 12 Naotham 0
```

حيث أن كل طلب يقوم tomcat بإنشاء ثريد خاص كما موضح بالصورة

without_virtual.jmx (C:\IntelliJ\ecommerce_backend1\src\main\java\com\example\demo\jmeter_files\without_virtual.jmx) - Apache JMeter (5.6.3)

Test Plan
Thread Group
HTTP Request
View Results Tree
Summary Report
View Results in Table

View Results in Table

Name: View Results in Table

Comments:

Write results to file / Read from file

Filename: Browse... Log/Display Only: ☐ Errors ☐ Successes Configure

Sample #	Start Time	Thread Name	Label	Sample Time(ms)	Status	Bytes	Sent Bytes	Latency	Connect Time(ms)
1	22:20:40.266	Thread Group 1-1	HTTP Request	1525	✓	211	237	1525	2
2	22:20:40.316	Thread Group 1-2	HTTP Request	2987	✓	211	237	2987	1
3	22:20:40.365	Thread Group 1-3	HTTP Request	4448	✓	211	237	4448	0
4	22:20:40.416	Thread Group 1-4	HTTP Request	5908	✓	211	237	5908	1
5	22:20:40.465	Thread Group 1-5	HTTP Request	7370	✓	211	237	7370	1
6	22:20:40.515	Thread Group 1-6	HTTP Request	8830	✓	211	237	8830	1
7	22:20:40.566	Thread Group 1-7	HTTP Request	10287	✓	211	237	10287	1
8	22:20:40.616	Thread Group 1-8	HTTP Request	11749	✓	211	237	11749	1
9	22:20:40.665	Thread Group 1-9	HTTP Request	13212	✓	211	237	13212	1
10	22:20:40.715	Thread Group 1-10	HTTP Request	14670	✓	211	237	14670	0
11	22:20:40.765	Thread Group 1-11	HTTP Request	16128	✓	211	237	16128	1
12	22:20:40.817	Thread Group 1-12	HTTP Request	17586	✓	211	237	17586	1
13	22:20:40.865	Thread Group 1-13	HTTP Request	19046	✓	211	237	19046	1
14	22:20:40.916	Thread Group 1-14	HTTP Request	20504	✓	211	237	20504	1
15	22:20:40.965	Thread Group 1-15	HTTP Request	21965	✓	211	237	21965	1
16	22:20:41.015	Thread Group 1-16	HTTP Request	23429	✓	211	237	23429	1
17	22:20:41.064	Thread Group 1-17	HTTP Request	24890	✓	211	237	24890	1
18	22:20:41.115	Thread Group 1-18	HTTP Request	26347	✓	211	237	26347	1
19	22:20:41.164	Thread Group 1-19	HTTP Request	27809	✓	211	237	27809	1
20	22:20:41.214	Thread Group 1-20	HTTP Request	29268	✓	211	237	29268	1

☐ Scroll automatically? ☐ Child samples? No of Samples: 20 Latest Sample: 29268 Average: 15347 Deviation: 8418

without_virtualjmx (C:\intelli\ecommerce_backend1\src\main\java\com\example\demo\meter_files\without_virtualjmx) - Apache JMeter (5.6.3)

File Edit Search Run Options Tools Help

00:00:30 0 0/20

Test Plan

- Thread Group
 - HTTP Request
 - View Results Tree
 - Summary Report
 - View Results in Table

Summary Report

Name: Summary Report

Comments:

Write results to file / Read from file

Filename: Log/Display Only: ☐ Errors ☐ Successes

Label	# Samples	Average	Min	Max	Std. Dev.	Error %	Throughput	Received KB/sec	Sent KB/sec	Avg. Bytes
HTTP Request	20	15397	1525	29268	8418.73	0.00%	39.7/min	0.14	0.15	211.0
TOTAL	20	15397	1525	29268	8418.73	0.00%	39.7/min	0.14	0.15	211.0

☐ Include group name in label? ☒ Save Table Header

without_virtualjmx (C:\intelli\ecommerce_backend1\src\main\java\com\example\demo\meter_files\without_virtualjmx) - Apache JMeter (5.6.3)

File Edit Search Run Options Tools Help

00:00:30 0 0/20

Test Plan

- Thread Group
 - HTTP Request
 - View Results Tree
 - Summary Report
 - View Results in Table

View Results Tree

Name: View Results Tree

Comments:

Write results to file / Read from file

Filename: Log/Display Only: ☐ Errors ☐ Successes

Search: ☐ Case sensitive ☐ Regular exp.

Text

Sampler result Request Response data

- HTTP Request
 - Thread Name: Thread Group 1-1
 - Sample Start: 2026-05-18 22:20:40 GMT+03:00
 - Load time: 1525
 - Connect Time: 2
 - Latency: 1525
 - Size in bytes: 211
 - Sent bytes: 237
 - Headers size in bytes: 162
 - Body size in bytes: 49
 - Sample Count: 1
 - Error Count: 0
 - Data type ("text/plain"): text
 - Response code: 200
 - Response message:
- HTTP Request
- HTTP Request
- HTTP Request
- HTTP Request
- HTTP Request
- HTTP Request
- HTTP Request
- HTTP Request
- HTTP Request
- HTTP Request
- HTTP Request
- HTTP Request
- HTTP Request
- HTTP Request
- HTTP Request
- HTTP Request
- HTTP Request
- HTTP Request
- HTTP Request
- HTTP Request

☐ Scroll automatically?

بعد التحسين :

```
File Edit View Navigate Code Refactor Build Run Tools Git Window Help ecommerce_backend1 - with_virtual.jmx
ecommerce_backend1 | src | main | java | com | example | demo | jmeter_files | with_virtual.jmx
Project | Local (3) | + | - | Add Configuration... | Git | LoadBalancerController.java | application.properties | docker-compose.yml | with_virtual.jmx | request5.jmx | LoadBalancerService.java | BatchCon | Maven
Terminal | Local (3) | + | - |
load-balancer | 2026-05-18T19:16:42.193Z | INFO 1 --- [demo] [mcmt-handler-57] | s.a.AnnotationAsyncExecutionInterceptor : More than one TaskExecutor bean found within the context, and none is named 'taskExecutor'. Mark one of them as primary or name it 'taskExecutor' (possibly as an alias) in order to use it for async processing: [batchTaskExecutor, taskScheduler]
load-balancer | 2026-05-18T19:16:42.198Z | INFO 1 --- [demo] [cTaskExecutor-1] | c.example.demo.aspect.PerformanceAspect : Started: ProductService.purchaseWithVirtual
load-balancer | 2026-05-18T19:16:42.198Z | INFO 1 --- [demo] [cTaskExecutor-1] | c.example.demo.aspect.PerformanceAspect : [PURCHASE] Started: purchaseWithVirtual | Thread: SimpleAsyncT
askExecutor-1 | 2026-05-18T19:16:42.236Z | INFO 1 --- [demo] [cTaskExecutor-2] | c.example.demo.aspect.PerformanceAspect : Started: ProductService.purchaseWithVirtual
load-balancer | 2026-05-18T19:16:42.236Z | INFO 1 --- [demo] [cTaskExecutor-2] | c.example.demo.aspect.PerformanceAspect : [PURCHASE] Started: purchaseWithVirtual | Thread: SimpleAsyncT
askExecutor-2 | 2026-05-18T19:16:42.286Z | INFO 1 --- [demo] [cTaskExecutor-3] | c.example.demo.aspect.PerformanceAspect : Started: ProductService.purchaseWithVirtual
load-balancer | 2026-05-18T19:16:42.286Z | INFO 1 --- [demo] [cTaskExecutor-3] | c.example.demo.aspect.PerformanceAspect : [PURCHASE] Started: purchaseWithVirtual | Thread: SimpleAsyncT
askExecutor-3 | 2026-05-18T19:16:42.336Z | INFO 1 --- [demo] [cTaskExecutor-4] | c.example.demo.aspect.PerformanceAspect : Started: ProductService.purchaseWithVirtual
load-balancer | 2026-05-18T19:16:42.336Z | INFO 1 --- [demo] [cTaskExecutor-4] | c.example.demo.aspect.PerformanceAspect : [PURCHASE] Started: purchaseWithVirtual | Thread: SimpleAsyncT
askExecutor-4 | 2026-05-18T19:16:42.387Z | INFO 1 --- [demo] [cTaskExecutor-5] | c.example.demo.aspect.PerformanceAspect : Started: ProductService.purchaseWithVirtual
load-balancer | 2026-05-18T19:16:42.387Z | INFO 1 --- [demo] [cTaskExecutor-5] | c.example.demo.aspect.PerformanceAspect : [PURCHASE] Started: purchaseWithVirtual | Thread: SimpleAsyncT
askExecutor-5 | 2026-05-18T19:16:42.435Z | INFO 1 --- [demo] [cTaskExecutor-6] | c.example.demo.aspect.PerformanceAspect : Started: ProductService.purchaseWithVirtual
load-balancer | 2026-05-18T19:16:42.435Z | INFO 1 --- [demo] [cTaskExecutor-6] | c.example.demo.aspect.PerformanceAspect : [PURCHASE] Started: purchaseWithVirtual | Thread: SimpleAsyncT
askExecutor-6 | 2026-05-18T19:16:42.486Z | INFO 1 --- [demo] [cTaskExecutor-7] | c.example.demo.aspect.PerformanceAspect : Started: ProductService.purchaseWithVirtual
load-balancer | 2026-05-18T19:16:42.486Z | INFO 1 --- [demo] [cTaskExecutor-7] | c.example.demo.aspect.PerformanceAspect : [PURCHASE] Started: purchaseWithVirtual | Thread: SimpleAsyncT
askExecutor-7 | 2026-05-18T19:16:42.535Z | INFO 1 --- [demo] [cTaskExecutor-8] | c.example.demo.aspect.PerformanceAspect : Started: ProductService.purchaseWithVirtual
load-balancer | 2026-05-18T19:16:42.535Z | INFO 1 --- [demo] [cTaskExecutor-8] | c.example.demo.aspect.PerformanceAspect : [PURCHASE] Started: purchaseWithVirtual | Thread: SimpleAsyncT
askExecutor-8 | 2026-05-18T19:16:42.585Z | INFO 1 --- [demo] [cTaskExecutor-9] | c.example.demo.aspect.PerformanceAspect : Started: ProductService.purchaseWithVirtual
load-balancer | 2026-05-18T19:16:42.585Z | INFO 1 --- [demo] [cTaskExecutor-9] | c.example.demo.aspect.PerformanceAspect : [PURCHASE] Started: purchaseWithVirtual | Thread: SimpleAsyncT
askExecutor-9 |
Tool Output | JMeter Preview | Git | TODO | Problems | Terminal | Build | Dependencies | Event Log | 1:1 | UTF-8 | 4 spaces | Nigham
```

with_virtual.jmx (C:\intelli\ecommerce_backend1\src\main\java\com\example\demo\jmeter_files\req2\with_virtual.jmx) - Apache JMeter (5.6.3)

File Edit Search Run Options Tools Help

00:00:15 0 0/10

Test Plan

- Thread Group
 - HTTP Request
 - View Results Tree
 - Summary Report
 - View Results in Table

View Results Tree

Name: View Results Tree

Comments:

Write results to file / Read from file

Filename: Browse... Log/Display Only: Errors Successes Configure

Search: Case sensitive Regular exp. Search Reset

Text

Sampler result: Request Response data

Response Body Response headers

SUCCESS: Purchased 1 x Mobile A55 | Remaining: 19

with_virtual.jmx (C:\intelli\ecommerce_backend1\src\main\java\com\example\demo\jmeter_files\req2\with_virtual.jmx) - Apache JMeter (5.6.3)

File Edit Search Run Options Tools Help

00:00:15 0 0/10

Test Plan

- Thread Group
 - HTTP Request
 - View Results Tree
 - Summary Report
 - View Results in Table

View Results in Table

Name: View Results in Table

Comments:

Write results to file / Read from file

Filename: Browse... Log/Display Only: Errors Successes Configure

Sample #	Start Time	Thread Name	Label	Sample Time(ms)	Status	Bytes	Sent Bytes	Latency	Connect Time(ms)
1	23:30:35.158	Thread Group 1-1	HTTP Request	1786	✓	211	234	1786	3
2	23:30:35.257	Thread Group 1-2	HTTP Request	3171	✓	211	234	3171	0
3	23:30:35.358	Thread Group 1-3	HTTP Request	4583	✓	211	234	4583	1
4	23:30:35.457	Thread Group 1-4	HTTP Request	5994	✓	211	234	5994	0
5	23:30:35.557	Thread Group 1-5	HTTP Request	7404	✓	211	234	7404	1
6	23:30:35.657	Thread Group 1-6	HTTP Request	8813	✓	211	234	8813	0
7	23:30:35.756	Thread Group 1-7	HTTP Request	10227	✓	211	234	10227	0
8	23:30:35.856	Thread Group 1-8	HTTP Request	11636	✓	211	234	11636	1
9	23:30:35.956	Thread Group 1-9	HTTP Request	13045	✓	211	234	13045	0
10	23:30:36.056	Thread Group 1-10	HTTP Request	14456	✓	211	234	14456	1

Label	# Samples	Average	Min	Max	Std. Dev.	Error %	Throughput	Received KB/sec	Sent KB/sec	Avg. Bytes
HTTP Request	10	8111	1786	14436	4047.68	0.00%	39.1/min	0.13	0.15	211.0
TOTAL	10	8111	1786	14436	4047.68	0.00%	39.1/min	0.13	0.15	211.0

الطلب الثالث :

1. المشكلة

في أنظمة التجارة الإلكترونية، عند إتمام عملية الشراء تحدث عدة مهام متسلسلة: تحديث المخزون، معالجة الدفع، إرسال إشعار للعميل، وإصدار فاتورة. في النهج التقليدي المتزامن، يظل المستخدم محجوباً في انتظار اكتمال جميع هذه المهام، بما فيها مهام ثانوية كإرسال البريد الإلكتروني التي لا تتطلب انتظاره. هذا يسبب زمن استجابة مرتفعاً جداً يصل إلى 5-8 ثوانٍ لكل طلب، مما يُضعف تجربة المستخدم ويُقيّد قدرة النظام على خدمة طلبات متعددة في آنٍ واحد.

جوهر المشكلة: كيف نُخرج المهام الثانوية من المسار الرئيسي للطلب بحيث يحصل المستخدم على استجابة فورية، مع ضمان تنفيذ هذه المهام لاحقاً بشكل موثوق؟

مثال

عند إتمام الشراء عملية الشراء النظام كان يقوم بإرسال الإشعارات بشكل متزامن مما يجبر المستخدم على الانتظار حتى تنتهي كل العمليات

مثال :

- مستخدم اشترى منتج
- النظام سجل الطلب
- النظام بدأ يرسل إيميل تأكيد : المستخدم ينتظر 3 ثواني
- النظام اصدر فاتورة : المستخدم لا يزال ينتظر

النتيجة : تجربة مستخدم سيئة وبطيئة في الاستجابة .

2.الحلول المقترحة

الحل الاول synchronous(بدون تحسين)

في هذا النهج تنفذ جميع المهام تسلسلياً في نفس Thread الرئيسي قبل الرد على المستخدم. يضمن هذا الحل البساطة والوضوح، لكنه يعاني من مشكلة جوهرية : المستخدم ينتظر 3 ثوانٍ أو أكثر لمجرد تأكيد الدفع، وإن فشل إرسال الإشعار فشل الطلب كله معه. تم استبعاد هذا الحل لأنه غير مقبول في بيئة إنتاجية تتعامل مع آلاف الطلبات المتزامنة.

الحل الثاني Async

يعتمد هذا الحل على تشغيل المهام الثانوية في Threads منفصلة داخل نفس التطبيق باستخدام Async@Spring. يحسن زمن الاستجابة بشكل ملحوظ، لكنه يحمل عيباً خطيراً : المهام تخزن في ذاكرة الـ JVM فقط، فإذا أعيد تشغيل الخادم أو انهار فجأة، تفقد جميع المهام المعلقة نهائياً دون أي إمكانية للاسترداد. هذا غير مقبول في سياق معالجة مدفوعات.

الحل الثالث Message Queue

يعتمد على إرسال المهام الثانوية كرسائل إلى قائمة انتظار مستدامة (Durable Queue) يعالجها Worker مستقل في الخلفية. هذا الحل يجمع بين السرعة في الاستجابة وضمان تنفيذ المهام، وهو المعيار الصناعي لهذا النوع من المشاكل.

3. سبب اختيار الـ Async

اختير RabbitMQ لأسباب تقنية محددة

أولاً:

الرسائل محفوظة على القرص (Durable Queues) ولا تفقد عند إعادة تشغيل الخادم.

ثانياً:

يعتمد على نظام التأكيد (ACK/NACK) ، بمعنى أن المستهلك يرسل تأكيداً للرسالة فقط بعد معالجتها بنجاح، وإن حدث خطأ تعود الرسالة تلقائياً للقائمة لإعادة المحاولة.

ثالثاً:

يوفر فصلاً تاماً بين المنتج والمستهلك (Decoupling) ، مما يعني أن فشل خدمة الإشعارات لا يؤثر بأي شكل على إتمام الطلب الأصلي

رابعاً:

يدعم التوسع الأفقي بإضافة consumers متعددين لنفس القائمة لرفع الإنتاجية عند الحاجة.

الآلية التنفيذية :

عند استدعاء (end point) /api/payment/async ، ينفذ المسار الرئيسي السريع فقط: التحقق من الرصيد، خصم المبلغ، حفظه في قاعدة البيانات، ثم إرسال رسالة إلى payment.queue في RabbitMQ. يعود الرد للمستخدم فوراً خلال أقل من 100 ms. في الخلفية وبشكل مستقل تماماً،

وعندها يبدأ الـ Consumer عبر @RabbitListener، يسحب الرسالة، ينفذ المعالجة الفعلية (3) ثوانٍ)، ويُرسَل تأكيد ACK عند النجاح.

تم تعريف قائمتين منفصلتين في RabbitMQConfig:

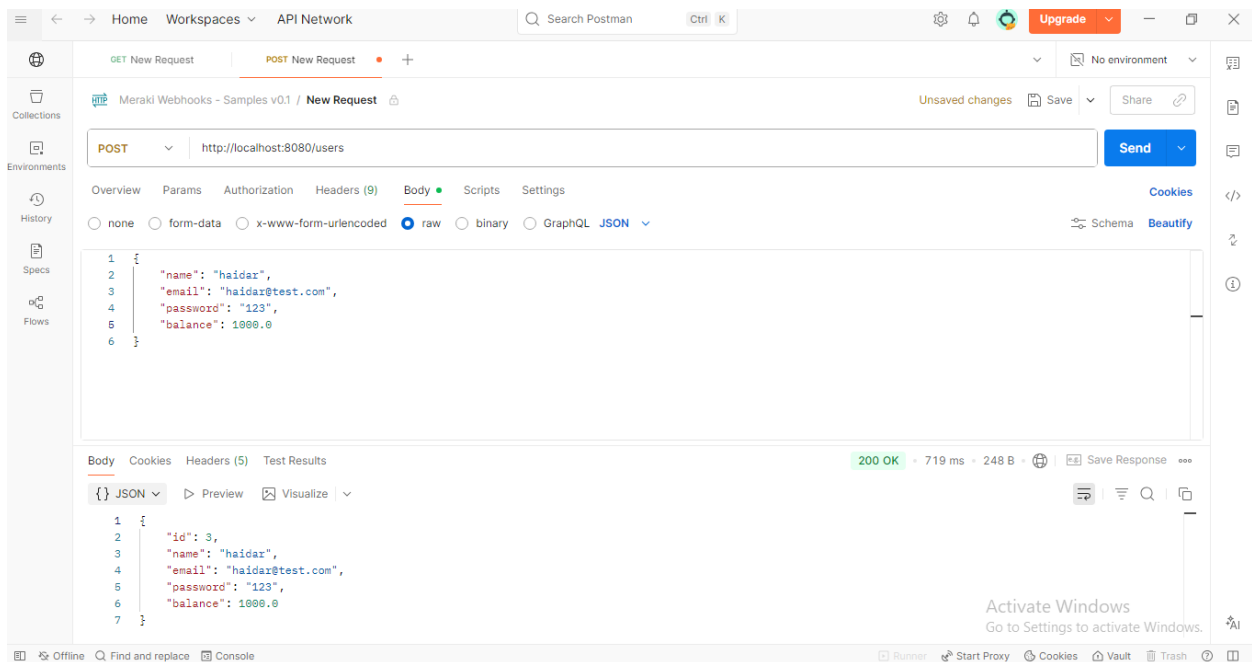
- **payment.queue** : لمعالجة تأكيدات الدفع
- **notification.queue** : لإرسال الإشعارات والفواتير

4. نتائج الاختبار بـ postman

سنجرب في هذا الاختبار نظام الدفع :

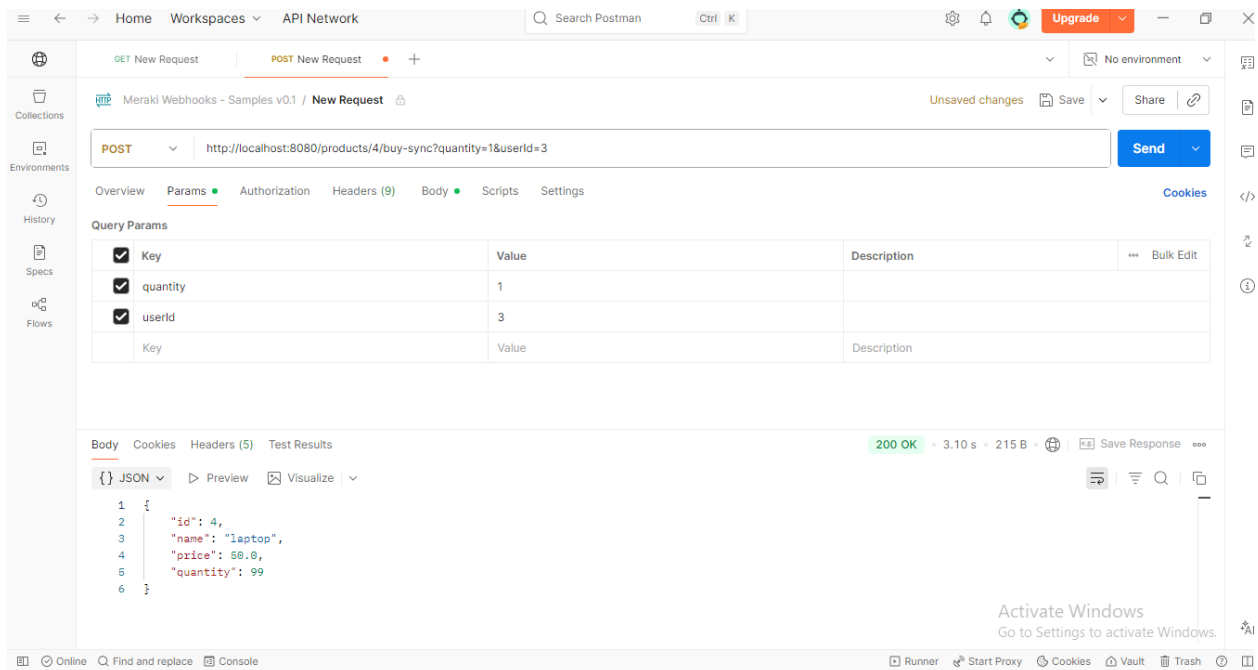
اولا :

سنضيف مستخدم جديد

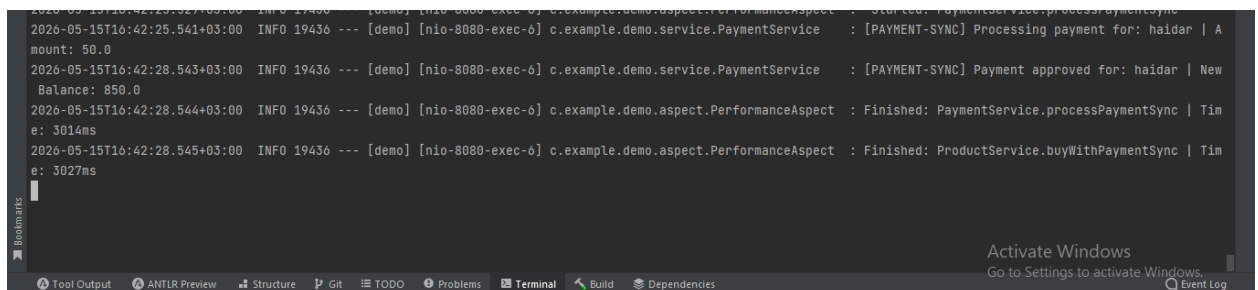


ثانيا :

سنجرب الدفع باستخدام الـ Blocking



قبل التحسين استغرق الامر 3 ثواني



[nio-8080-exec-6] PaymentService : [PAYMENT-SYNC] Processing payment for: haidar | Amount: 50.0

بدأت المعالجة... والمستخدم لا زال ينتظر.

[PAYMENT-SYNC] Payment approved for: haidar | New Balance: 850.0

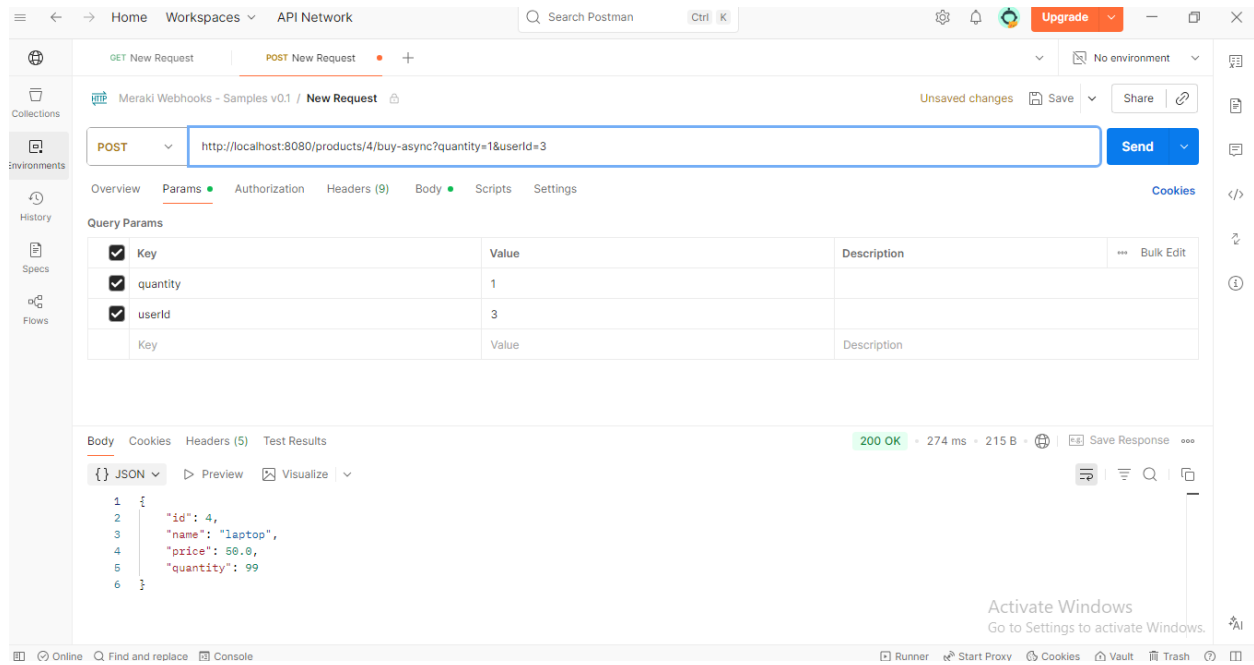
اكتملت المعالجة، الرصيد صار 850.

Finished: ProductService.buyWithPaymentSync | Time: 3027ms

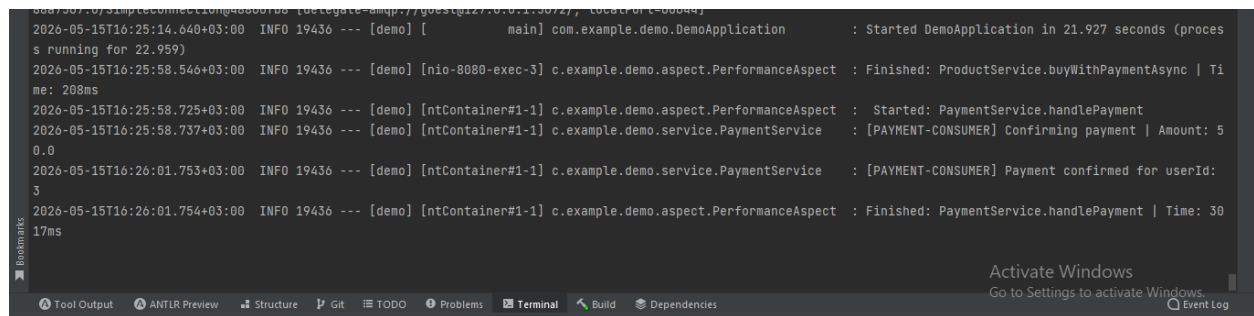
لمستخدم انتظر 3027ms كاملة قبل ما يحصل على الرد.

ثالثا :

سنجرب الدفع الان باستخدام ال non blocking



بعد التحسين استغرق 274ms اي ان الرد كان سريع وهذا يوضح الفرق



هنا صورة من التيرمينال عن ال logs سنشرح شو صار بالضبط :

[nio-8080-exec-3] PerformanceAspect : Finished :
ProductService.buyWithPaymentAsync | Time: 208ms

لصار انو المستخدم طلب الشراء فالطلب اكتمل وعاد للمستخدم خلال 208ms

[ntContainer#1-1] PerformanceAspect : Started:
PaymentService.handlePayment [ntContainer#1-1] PaymentService :
[PAYMENT-CONSUMER] Confirming payment | Amount: 50.0

هنا بدأ ال consumer يشتغل بشكل مستقل في thread مختلف تماما

[ntContainer#1-1] PaymentService : [PAYMENT-CONSUMER] Payment
confirmed for userId: 3

[ntContainer#1-1] PerformanceAspect : Finished:
PaymentService.handlePayment | Time: 3017ms

ال consumer خلص شغله بعد 3 ثواني يعني (لمعالجة الفعلية أخذت 3017ms لكن المستخدم ما انتظرها) .

الطلب الرابع :

1. المشكلة

في أنظمة التجارة الإلكترونية، تتراكم كميات ضخمة من بيانات المبيعات يومياً. في نهاية كل يوم يحتاج النظام لمعالجة هذه السجلات، كحساب التقارير اليومية وتحديث حالة المبيعات وعند معالجة كميات كبيرة من بيانات المبيعات دفعة واحدة يحدث ما يلي :

- تحميل جميع السجلات لل RAM دفعة واحدة يؤدي الى انهيار النظام .
- معالجة السجلات بشكل تسلسلي تستغرق وقتاً طويلاً جداً .

على سبيل المثال كل سجل يأخذ 1 ثانية ولدينا 10 ملايين سجل المعالجة التسلسلية تأخذ 115 يوم

2.الحلول المقترحة

الحل الاول معالجة كل البيانات دفعة واحدة

يعتمد على جلب جميع السجلات بأمر findAll() ومعالجتها في حلقة واحدة. بسيط في الكتابة لكنه كارثي على الأداء، إذ يحمل كل شيء في الذاكرة (RAM) مرة واحدة. مع 100,000 سجل يستهلك هذا

الحل مئات الميغابايت من الذاكرة، ويجمد قاعدة البيانات لفترة طويلة، ويرفع احتمالية الانهيار الكامل. تم استبعاده لأنه غير قابل للتوسع أبدا .

الحل الثاني Fixed – size chunking

يعتمد على تقسيم البيانات إلى دفعات بحجم ثابت محدد مسبقا، مثلا 100 سجل لكل دفعة بغض النظر عن العدد الكلي.

أبسط في التنفيذ من الحل المختار، لكنه يحمل مشكلة جوهرية: الحجم الثابت لا يتكيف مع تغير البيانات. إن كان عندك 500 سجل اليوم و500,000 سجل غدا، نفس الحجم الثابت يسبب إما كثرة استعلامات مع البيانات القليلة أو بطء مع البيانات الكثيرة. تم استبعاده لصالح الحجم الديناميكي الذي يتكيف تلقائيا مع حجم البيانات الفعلي.

الحل الثالث Dynamic Partitioning + spring Batch (المختار)

يعتمد هذا الحل على **Spring Batch** الذي يوفر بنية جاهزة لمعالجة البيانات الضخمة بطريقة منظمة وموثوقة، مع تقسيم العمل بشكل ذكي إلى أجزاء قابلة للتنفيذ بالتوازي (هو مخصص لمعالجة ال batch processing في ال spring Boot).

- يتم تقسيم البيانات إلى Partitions (نطاقات IDs)
- كل Partition يتم معالجته بواسطة Worker Thread مستقل .
- داخل كل Worker يتم استخدام Chunk-oriented Processing

3.سبب اختيار Dynamic Partitioning

1. حجم Chunk ديناميكي (Dynamic Chunk Size) :

يتم حساب حجم الدفعة حسب عدد السجلات الفعلي:

- حتى $10 \leq 100$
- حتى $100 \leq 1000$
- حتى $500 \leq 10,000$
- أكثر من ذلك $1000 \leq$

2. تقليل الضغط على الذاكرة (Memory Efficient) :

Spring Batch لا يحمل كل البيانات دفعة واحدة، بل يعالج كل Chunk بشكل مستقل

(Read → Process → Write)

3. تحسين الأداء عبر التوازي (Parallel Processing) :

باستخدام TaskExecutorPartition يتم تشغيل عدة Workers بالتوازي، مما يقلل وقت التنفيذ بشكل كبير.

4. إدارة الحالة (Job State Management) :

Spring Batch يقوم بتخزين حالة الـ Job في قاعدة البيانات عبر:

- JobRepository
- JobExplorer

وهذا يسمح باستكمال التنفيذ بعد الانقطاع (Restartability) .

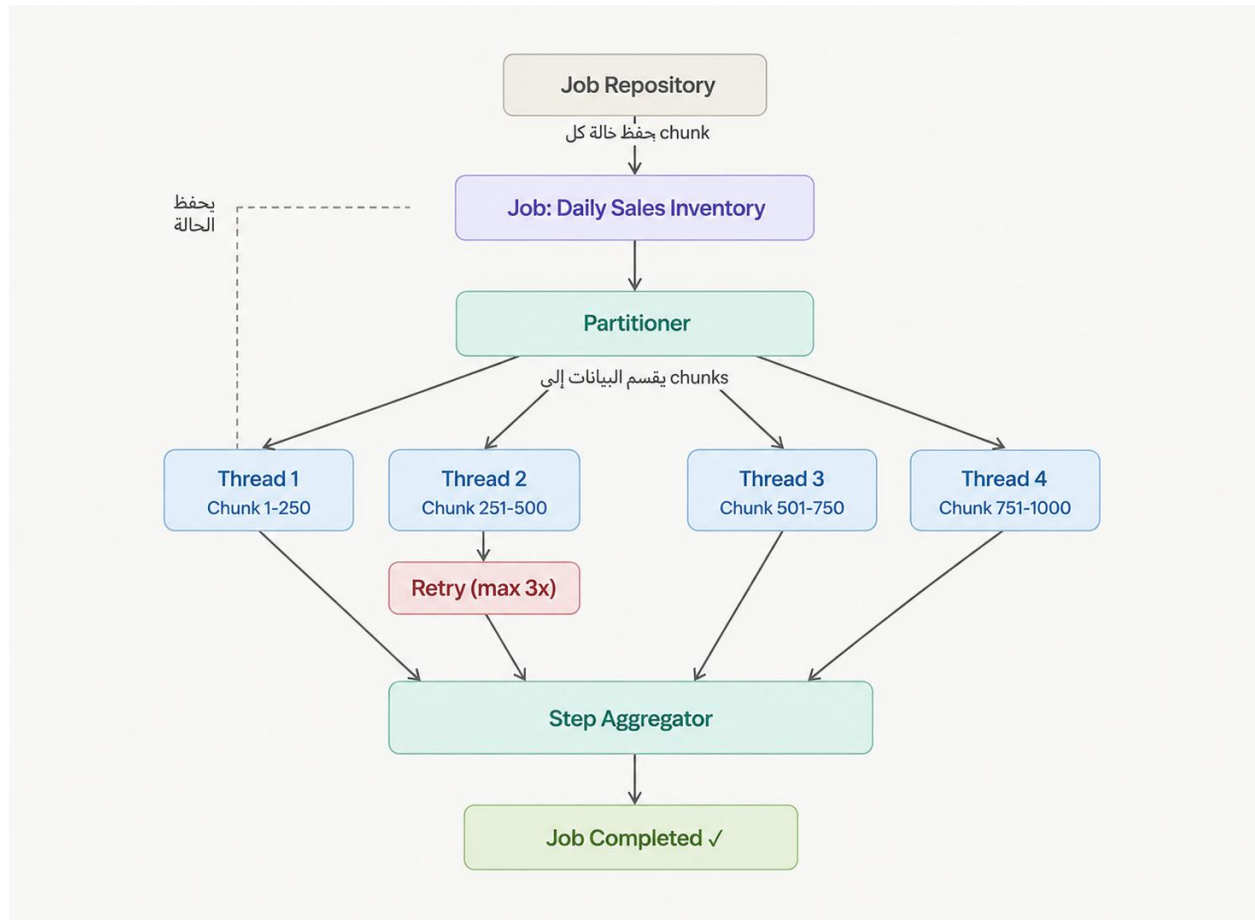
5. Fault Tolerance

الـ Fault Tolerance ييحفظ حالة كل step في الـ DB جدول إذا مات الـ container في المنتصف، لما يرجع بيعرف من وين يكمل

يدعم:

- retry لإعادة محاولة معالجة السجل عند الفشل
- skip لتجاوز السجلات الخاطئة دون إيقاف العملية كاملة

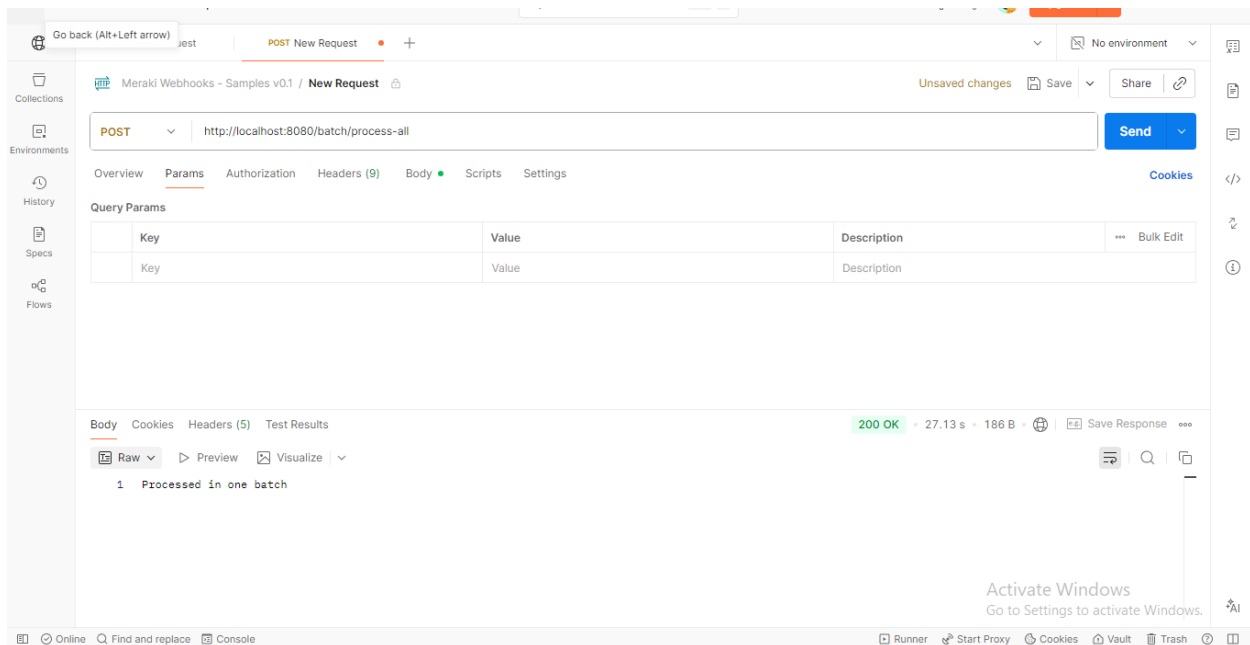
6. هذا Diagram يوضح طريقة عملها :



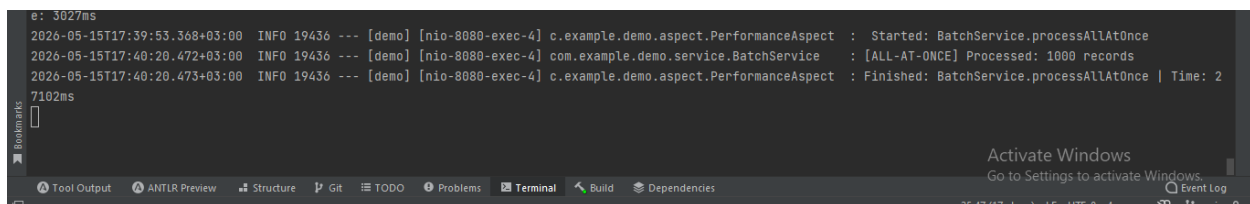
4. نتائج الاختبار بـ postman

تم اختبار النظام في postman على 1000 records:

اولا : قبل التحسين



اسغرق الأمر 27 ثانية

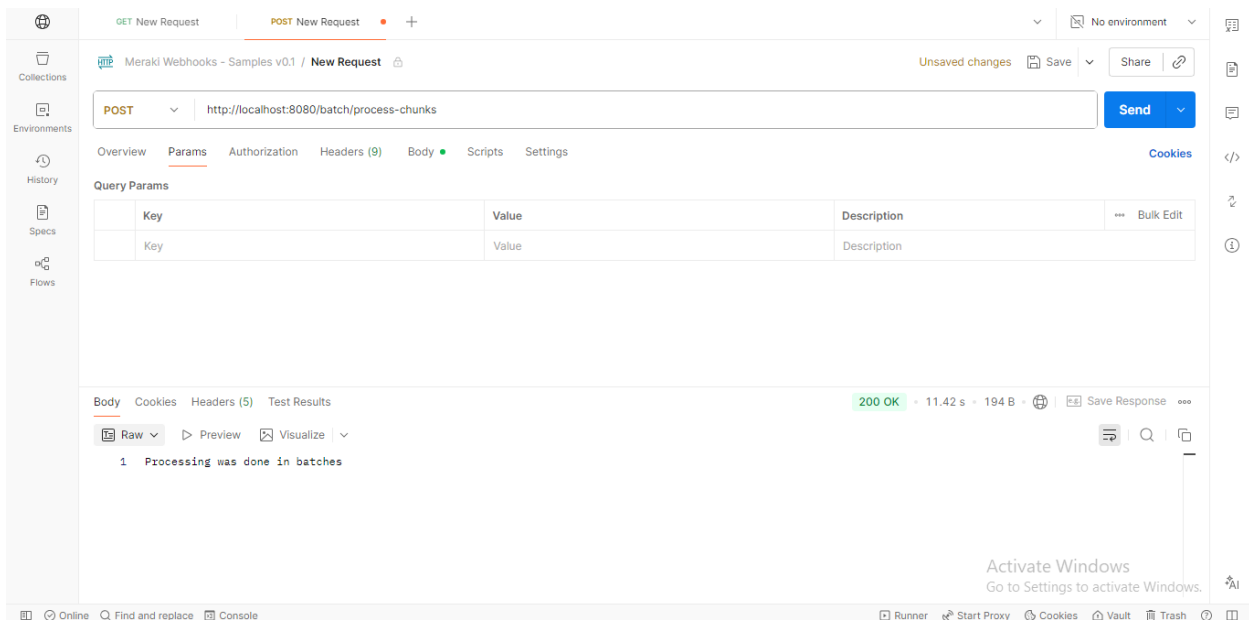


Started: BatchService.processAllAtOnce
هذا يعني أن النظام بدأ تشغيل processAllAtOnce ()

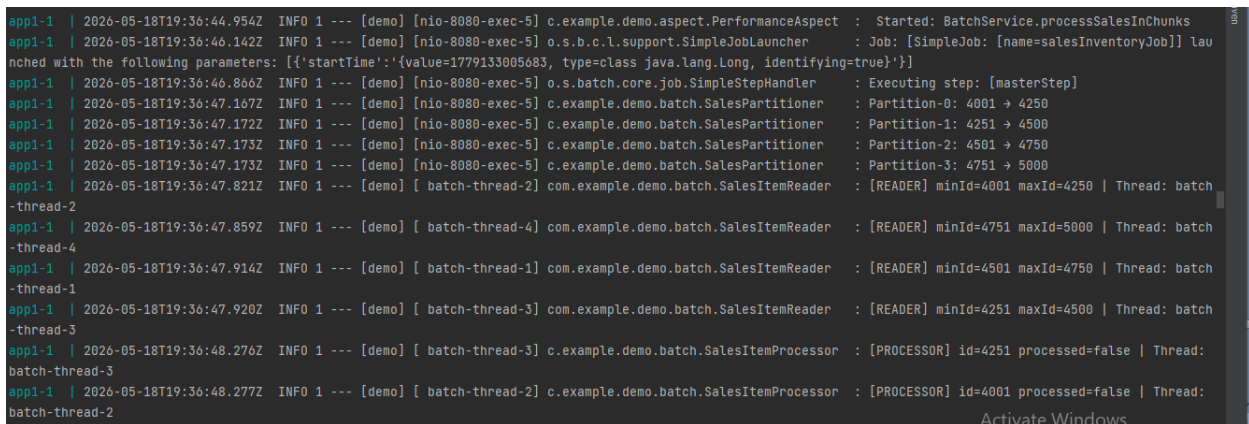
[ALL-AT-ONCE] Processed: 1000 records
هنا النظام عالج 1000 سجل (record) دفعة واحدة.

Finished: BatchService.processAllAtOnce | Time: 27102ms
وقت التنفيذ 27

ثانيا : بعد التحسين



استغرق الامر 11 ms وهذا الوقت على بيانات كبيرة يفرق جدا في الاداء



Executing step: [masterStep]
 هنا تم تشغيل ال masterstep وهو المسؤول عن تقسيم البيانات partition

تقسيم بيانات المبيعات إلى أجزاء (Ranges)

بدل ما يعالج كل البيانات دفعة واحدة، قسمها إلى 4 أجزاء:

- Partition 0: من 4001 إلى 4250
- Partition 1: من 4251 إلى 4500

- Partition 2: من 4501 إلى 4750
- Partition 3: من 4751 إلى 5000

batch-thread-1
batch-thread-2
batch-thread-3
batch-thread-4

هون واضح انو النظام شغل 4 thread بالتوازي كل Thread مسؤول عن Partition معينة.

وبعدين صار يطبق المراحل تبغو (Read → Process → Write)

ملخص بسيط :

تم تنفيذ Batch Processing لمعالجة بيانات المبيعات الضخمة باستخدام أسلوب Chunk Processing لتحسين الأداء. بدلاً من تحميل جميع السجلات مرة واحدة، يتم تقسيم البيانات إلى دفعات صغيرة (Chunks)، حيث تتم معالجة كل دفعة بشكل مستقل. كما تم تشغيل العملية كـ Background Job باستخدام @Scheduled، ليتم تنفيذها تلقائياً يومياً الساعة الثانية صباحاً، مما يقلل الضغط على النظام ويحسن استهلاك الذاكرة وزمن التنفيذ. عند معالجة 1000 سجل، انخفض زمن التنفيذ من حوالي 27 ثانية عند استخدام processAllAtOnce () إلى حوالي 11 ثانية باستخدام Chunks.

الطلب الخامس:

مع ازدياد عدد المستخدمين والطلبات في الأنظمة أصبح الاعتماد على Server واحد غير كافٍ لتقديم أداء مستقر وفعال، لأن ذلك يؤدي إلى مشاكل عديدة مثل:

- زيادة الضغط على الخادم
- بطء الاستجابة
- اختناق النظام عند ارتفاع عدد الطلبات
- عدم استغلال الموارد بشكل متوازن
- احتمالية توقف الخدمة بالكامل عند فشل السيرفر

الهدف من هذا الطلب لم يكن فقط توزيع الطلبات على عدة خوادم وإنما بناء آلية تراعي:

- الحمل الحالي على كل خادم
- قدرة كل خادم على المعالجة
- الأداء تحت الضغط
- التزامن بين الطلبات Concurrent Requests
- قابلية التوسع مستقبلاً

المشكلة لم تكن فقط إرسال الطلبات إلى عدة Servers ، وإنما: كيف يمكن توزيع الطلبات بشكل ذكي وعادل مع مراعاة حالة كل سيرفر وقدرته الفعلية؟ لأن السيرفرات في الواقع ليست متساوية دائماً فقد يكون:

- أحد السيرفرات أقوى من غيره
- أحدها مشغولاً حالياً بعدد كبير من الطلبات
- أحدها يمتلك موارد معالجة أكبر

وبالتالي فإن توزيع الطلبات بشكل عشوائي أو ثابت قد يؤدي إلى:

- ضغط زائد على بعض السيرفرات
- ضعف استغلال السيرفرات الأقوى
- انخفاض الأداء العام للنظام

ولتحقيق ذلك قمنا باستخدام :

- Docker لمحاكاة عدة سيرفرات مستقلة
- خوارزمية Hypaired تجمع بين :
 - Least Connections
 - Weighted Distribytion

دراسة الخوازميات الممكنة :

قبل اختيار الحل النهائي قمنا بدراسة عدة خوارزميات Load Balancing :

الحلول المقترحة :

أولاً : Random Load Balancing

في هذه الطريقة يتم اختيار السيرفر بشكل عشوائي لكل طلب

مثال:

- Request 1 → app1
- Request 2 → app3
- Request 3 → app1

لماذا لم نختار هذا الحل :

لأن :

- التوزيع غير مستقر
- لا يراعي الضغط الحالي على السيرفر
- لا يراعي قوة السيرفرات
- قد يحصل سيرفر ضعيف على عدد كبير من الطلبات

لذلك اعتبرناه غير مناسب لنظامنا.

ثانياً : Round Robin

في هذه الطريقة يتم توزيع الطلبات بالتتابع بين السيرفرات : app1 → app2 → app3 → app1
لماذا لم نستخدمه :

رغم أن هذه الطريقة تُعدّ أفضل من Random إلا أنها تفترض أن جميع السيرفرات متساوية أي جميعها نفس القدرة بينما في نظامنا :

- بعض السيرفرات أقوى من غيرها
- بعض السيرفرات تمتلك قدرة معالجة أكبر

وبالتالي توزيع نفس عدد الطلبات على جميع السيرفرات يُعتبر قرار غير واقعي

ثالثاً : Weighted Round Robin

في هذه الطريقة يحصل السيرفر الأقوى على عدد أكبر من الطلبات اعتماداً على قيمة ال weight
مثال : بما أننا وضعنا لل app2 أعلى وزن 3 لذلك يحصل على طلبات أكثر
ولكن لماذا لم يكن كافياً أيضاً ؟

لأن هذا الحل يراعي فقط قوة السيرفر لكنه لا يراعي الحمل الحالي
قد يكون السيرفر قوياً لكنه مشغول حالياً بعدد كبير من الطلبات وبالتالي قد يستمر النظام بإرسال Request إضافية إليه رغم أنه مشغول .
وهذا قد يؤدي إلى:

- زيادة الضغط على السيرفر الأقوى
- تأخير بعض الطلبات
- اختلال التوازن أثناء التشغيل الفعلي

قرارنا النهائي :

بعد دراسة الحلول السابقة قررنا استخدام : Weighted Least Connections

وهي خوارزمية Hypired تجمع بين :

التقنية	الهدف
---------	-------

التقنية	الهدف
Least Connections	معرفة أقل السيرفرات انشغالاً
Weight	مراعاة قوة السيرفر

نظامنا يتكوّن من :

المكوّن	الوظيفة
Load Balancer	استقبال الطلبات واختيار أفضل Server
Worker Servers	تنفيذ الطلبات الفعلية
Docker Containers	محاكاة السيرفرات المستقلة
Logging System	مراقبة الأداء وتتبع التنفيذ

بنية النظام (System Architecture) :

قمنا ببناء عدّة Containers حيث يحتوي النظام على :

الدور	Container
Load Balancer	lb
Worker Instance	app1
Worker Instance	app2
Worker Instance	app3
قاعدة البيانات	mysql
التخزين المؤقت Cache	redis

لماذا استخدمنا Docker في هذا الطلب بسبب توفّر عدّة ميزات منها:

- تشغيل عدّة Instances بسهولة
- عزل الخدمات عن بعضها
- إنشاء شبكة داخلية بين السيرفرات
- سهولة الاختبار والتوسع
- إمكانية إعادة تشغيل النظام بالكامل بأمر واحد

لماذا قمنا بإنشاء عدّة Worker Servers؟

في الأنظمة الحقيقية لا يتم الاعتماد على سيرفر واحد فقط، لأن ذلك يؤدي إلى:

- زيادة الضغط
- نقطة فشل وحيدة Single Point of Failure
- انخفاض الأداء عند زيادة المستخدمين

لذلك قمنا بإنشاء عدّة instance :

Server	Weight
app1	1
app2	3
app3	2

كل Server يمثل سيرفر مختلف بقدرات معالجة مختلفة.

لماذا استخدمنا نفس التطبيق لكل الـ **Instances** ؟

قمنا باستخدام نفس نسخة التطبيق لكل السيرفرات لكن قمنا بتغيير السلوك عبر Environment Variables عن طريق :

▪ APP_ROLE

▪ APP_NAME

وهذا سمح لنا بـ :

▪ تقليل تكرار الكود

▪ سهولة التوسّع

▪ تشغيل عدّة أدوار بنفس التطبيق

لماذا استخدمنا **Weight** ؟

الـ Weight يمثل قدرة السيرفر على التحمّل فمثلاً:

▪ app1 : سيرفر ضعيف

▪ app2 : سيرفر قوي

▪ app3 : سيرفر متوسط

وبالتالي لا يحصل جميع السيرفرات على نفس عدد الطلبات لأن السيرفر الأقوى يجب أن يتحمّل عدد أكبر من الـ Request .

كيف تعمل الخوارزمية :

قمنا بحساب قيمة لكل سيرفر :

$$\text{Score} = \text{activeConnections} / \text{weight}$$

ثم نقوم باختيار السيرفر ذو أقل قيمة (Score) حيث:

▪ activeConnections : عدد الطلبات الحالية

▪ weight : قدرة السيرفر

مثال عملي بسيط قبل عرض صور الاختبار :

Server	Active	Weight	Score
app1	1	1	1.0

Server	Active	Weight	Score
app2	2	3	0.66
app3	1	2	0.5

رغم أن app2 لديه طلبات أكثر من app1 إلا أن قدرته أعلى لذلك يبقى مؤهلاً لاستقبال طلبات إضافية كما عالجت حالات تعادل ل Score (Tie Handling) :
 في بعض الحالات قد تمتلك عدة سيرفرات نفس ل Score لذلك قمنا بإضافة قاعدة إضافية وهي :
 إذا تساوت الـ Scores يتم اختيار السيرفر الأعلى Weight لأن السيرفر الأقوى أكثر قدرة على تحمل الضغط المستقبلي

لماذا اعتبرناه الحل الأفضل :

لأن الخوارزمية :

- منع الضغط الزائد على السيرفرات الضعيفة
- دعم العمل تحت الضغط العالي
- تمنع اختناق السيرفرات الضعيفة
- توزيع الأحمال بشكل متوازن
- استغلال السيرفرات القوية بشكل أفضل

: LoadBalancingService

قمنا ببناء Service مسؤولة عن :

- تحميل السيرفرات
- حساب الـ Scores
- اختيار أفضل سيرفر
- تحديث عدد الاتصالات الحالية
- تحرير السيرفر بعد انتهاء الطلب

: لماذا استخدمنا synchronized

النظام لدينا يتعامل مع عدة طلبات متزامنة وهذا قد يسبب Race Condition حيث يمكن لعدة Threads اختيار نفس السيرفر بنفس اللحظة إذا لم تتم حماية عملية الاختيار

وهكذا نكون قد ضمنا :

- سلامة البيانات
- منع التضارب
- الحفاظ على دقة عدد الاتصالات الحالية لكل Server

لماذا لم نستخدم Pessimistic Locking من الطلب الأول :

لأنه مخصص لحماية البيانات داخل قاعدة البيانات DB بينما مشكلتنا الحالية In-Memory Concurrency أي أن البيانات موجودة داخل RAM وليس داخل Database لذلك استخدمنا في هذا الطلب synchronized

لماذا لم نستخدم ReentrantLock أو AtomicInteger ؟

قمنا بدراسة حلول أخرى مثل:

- ReentrantLock
- AtomicInteger
- Concurrent Collections

لكننا لم نستخدمها لأن:

- النظام صغير نسبياً
- synchronized أبسط وأوضح
- يحقق Thread Safety بشكل كافٍ

ما الفرق الـ Load Balancer عن الـ Workers ؟

- الـ Worker مسؤول عن تنفيذ الطلبات الفعلية ومعالجة العمليات داخل كل Instance
- أما الـ Load Balancer مسؤول عن استقبال الطلبات القادمة واختيار أفضل سيرفر اعتماداً على خوارزمية :

Weighted Least Connections

حيث:

- **LoadBalancerService** مسؤول عن :
 - حساب الحمل الحالي على السيرفرات
 - تطبيق خوارزمية الاختيار
 - توجيه الطلب إلى أفضل Instance
- **Worker Instances** مسؤولة عن:

- تنفيذ الطلبات الفعلية
- معالجة العمليات
- التعامل مع قاعدة البيانات
- إرجاع النتائج للمستخدم

وبالتالي حافظ النظام على الفصل الوظيفي بين:

- منطق توزيع الحمل
- ومنطق تنفيذ العمليات

وأيضاً لقد استعنا بـ فايل logger الذي استخدمناه لتحقيق مبدأ لـ AOP من أجل أن يسجل النظام :

- السيرفر المختار
- عدد الاتصالات الحالية
- الـ Score لكل سيرفر
- زمن التنفيذ
- اسم الثريد
- تحرير السيرفر بعد انتهاء الطلب

النتيجة النهائية التي حصلنا عليها :
أصبح نظامنا يتميز ب:

- توزيع أحمال ذكي
- دعم الـ Concurrency
- مراعاة قوة السيرفرات
- منع Race Conditions
- قابلية التوسع
- مراقبة الأداء
- محاكاة بيئة Distributed حقيقية

كما أن الحل الذي اعتمدناه يتفوق على:

- Random
- Round Robin
- Weighted Round Robin

لأنه يعتمد على:

- الحمل الحقيقي الحالي
- قوة السيرفر
- التوازن الديناميكي أثناء التشغيل

ولذلك اعتبرنا أن: Weighted Least Connections

هو القرار الهندسي الأنسب لتحقيق متطلبات مشروعنا

الاختبار : قبل وبعد التحسين :

قبل التحسين: الضغط كامل على السيرفر الاول

app1-1		2026-05-18T19:50:08.534Z	INFO	1	---	[demo]	[omcat-handler-3]	c.e.d.c.WithoutLoadBalancerController	: Request handled ONLY by app1
app1-1		2026-05-18T19:50:08.585Z	INFO	1	---	[demo]	[omcat-handler-4]	c.e.d.c.WithoutLoadBalancerController	: Request handled ONLY by app1
app1-1		2026-05-18T19:50:08.634Z	INFO	1	---	[demo]	[omcat-handler-5]	c.e.d.c.WithoutLoadBalancerController	: Request handled ONLY by app1
app1-1		2026-05-18T19:50:08.686Z	INFO	1	---	[demo]	[omcat-handler-6]	c.e.d.c.WithoutLoadBalancerController	: Request handled ONLY by app1
app1-1		2026-05-18T19:50:08.733Z	INFO	1	---	[demo]	[omcat-handler-7]	c.e.d.c.WithoutLoadBalancerController	: Request handled ONLY by app1
app1-1		2026-05-18T19:50:08.786Z	INFO	1	---	[demo]	[omcat-handler-8]	c.e.d.c.WithoutLoadBalancerController	: Request handled ONLY by app1
app1-1		2026-05-18T19:50:08.835Z	INFO	1	---	[demo]	[omcat-handler-9]	c.e.d.c.WithoutLoadBalancerController	: Request handled ONLY by app1
app1-1		2026-05-18T19:50:08.887Z	INFO	1	---	[demo]	[mcat-handler-10]	c.e.d.c.WithoutLoadBalancerController	: Request handled ONLY by app1
app1-1		2026-05-18T19:50:08.935Z	INFO	1	---	[demo]	[mcat-handler-11]	c.e.d.c.WithoutLoadBalancerController	: Request handled ONLY by app1
app1-1		2026-05-18T19:50:08.984Z	INFO	1	---	[demo]	[mcat-handler-12]	c.e.d.c.WithoutLoadBalancerController	: Request handled ONLY by app1
app1-1		2026-05-18T19:50:09.035Z	INFO	1	---	[demo]	[mcat-handler-13]	c.e.d.c.WithoutLoadBalancerController	: Request handled ONLY by app1
app1-1		2026-05-18T19:50:09.085Z	INFO	1	---	[demo]	[mcat-handler-14]	c.e.d.c.WithoutLoadBalancerController	: Request handled ONLY by app1
app1-1		2026-05-18T19:50:09.134Z	INFO	1	---	[demo]	[mcat-handler-15]	c.e.d.c.WithoutLoadBalancerController	: Request handled ONLY by app1
app1-1		2026-05-18T19:50:09.184Z	INFO	1	---	[demo]	[mcat-handler-16]	c.e.d.c.WithoutLoadBalancerController	: Request handled ONLY by app1
app1-1		2026-05-18T19:50:09.234Z	INFO	1	---	[demo]	[mcat-handler-17]	c.e.d.c.WithoutLoadBalancerController	: Request handled ONLY by app1
app1-1		2026-05-18T19:50:09.287Z	INFO	1	---	[demo]	[mcat-handler-18]	c.e.d.c.WithoutLoadBalancerController	: Request handled ONLY by app1
app1-1		2026-05-18T19:50:09.334Z	INFO	1	---	[demo]	[mcat-handler-19]	c.e.d.c.WithoutLoadBalancerController	: Request handled ONLY by app1

without_load.jmx (C:\intelli\ecommerce_backend\src\main\java\com\example\demo\jmeter_files\without_load.jmx) - Apache JMeter (5.6.3)

00:00:02 0 0/20

Test Plan
Thread Group
HTTP Request
View Results Tree
View Results in Table
Aggregate Report

View Results in Table

Name: View Results in Table

Comments:

Write results to file / Read from file

Filename: Log/Display Only: ☐ Errors ☐ Successes

Sample #	Start Time	Thread Name	Label	Sample Time(ms)	Status	Bytes	Sent Bytes	Latency	Connect Time(ms)
1	22:40:52.910	Thread Group 1-2	HTTP Request	2060	✓	184	141	2060	1
2	22:40:52.861	Thread Group 1-1	HTTP Request	2109	✓	184	141	2109	1
3	22:40:52.960	Thread Group 1-3	HTTP Request	2010	✓	184	141	2010	1
4	22:40:53.009	Thread Group 1-4	HTTP Request	2007	✓	184	141	2007	1
5	22:40:53.062	Thread Group 1-5	HTTP Request	2008	✓	184	141	2008	1
6	22:40:53.110	Thread Group 1-6	HTTP Request	2006	✓	184	141	2006	1
7	22:40:53.160	Thread Group 1-7	HTTP Request	2006	✓	184	141	2006	0
8	22:40:53.211	Thread Group 1-8	HTTP Request	2005	✓	184	141	2005	0
9	22:40:53.260	Thread Group 1-9	HTTP Request	2006	✓	184	141	2006	0
10	22:40:53.311	Thread Group 1-10	HTTP Request	2006	✓	184	141	2006	0
11	22:40:53.362	Thread Group 1-11	HTTP Request	2005	✓	184	141	2005	0
12	22:40:53.410	Thread Group 1-12	HTTP Request	2006	✓	184	141	2006	0
13	22:40:53.460	Thread Group 1-13	HTTP Request	2005	✓	184	141	2005	1
14	22:40:53.509	Thread Group 1-14	HTTP Request	2007	✓	184	141	2007	1
15	22:40:53.560	Thread Group 1-15	HTTP Request	2007	✓	184	141	2007	1
16	22:40:53.610	Thread Group 1-16	HTTP Request	2005	✓	184	141	2005	1
17	22:40:53.661	Thread Group 1-17	HTTP Request	2008	✓	184	141	2008	2
18	22:40:53.710	Thread Group 1-18	HTTP Request	2006	✓	184	141	2006	1
19	22:40:53.761	Thread Group 1-19	HTTP Request	2007	✓	184	141	2007	1
20	22:40:53.822	Thread Group 1-20	HTTP Request	2007	✓	184	141	2007	1

☐ Scroll automatically? ☐ Child samples? No of Samples: 20 Latent Sample: 2007 Average: 2014 Deviation: 24

without_load.jmx (C:\intelli\ecommerce_backend\src\main\java\com\example\demo\jmeter_files\without_load.jmx) - Apache JMeter (5.6.3)

00:00:02 0 0/20

Test Plan
Thread Group
HTTP Request
View Results Tree
View Results in Table
Aggregate Report

View Results Tree

Name: View Results Tree

Comments:

Write results to file / Read from file

Filename: Log/Display Only: ☐ Errors ☐ Successes

Search: ☐ Case sensitive ☐ Regular exp.

Text

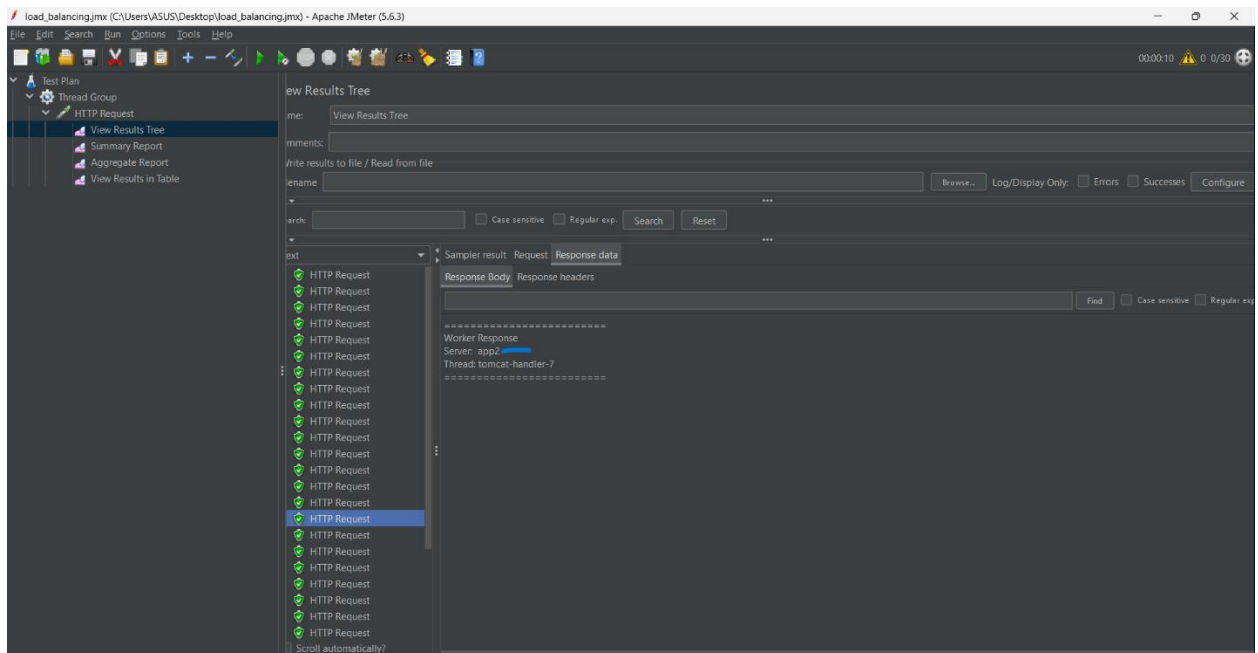
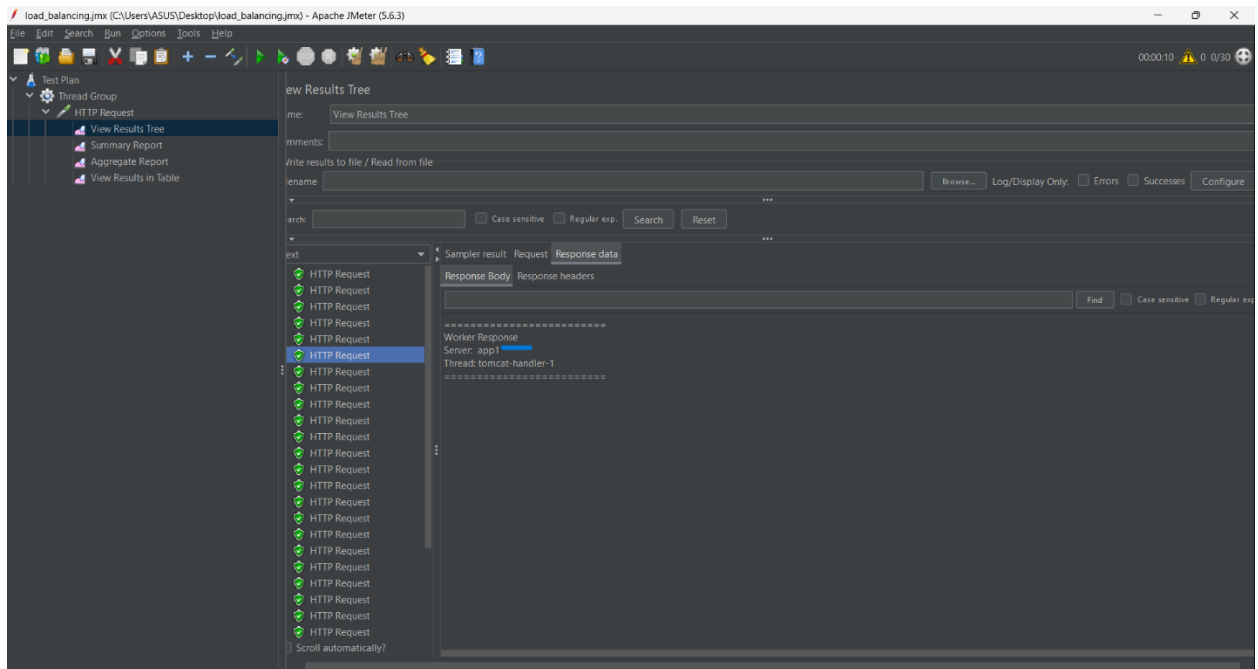
Sampler result Request Response data

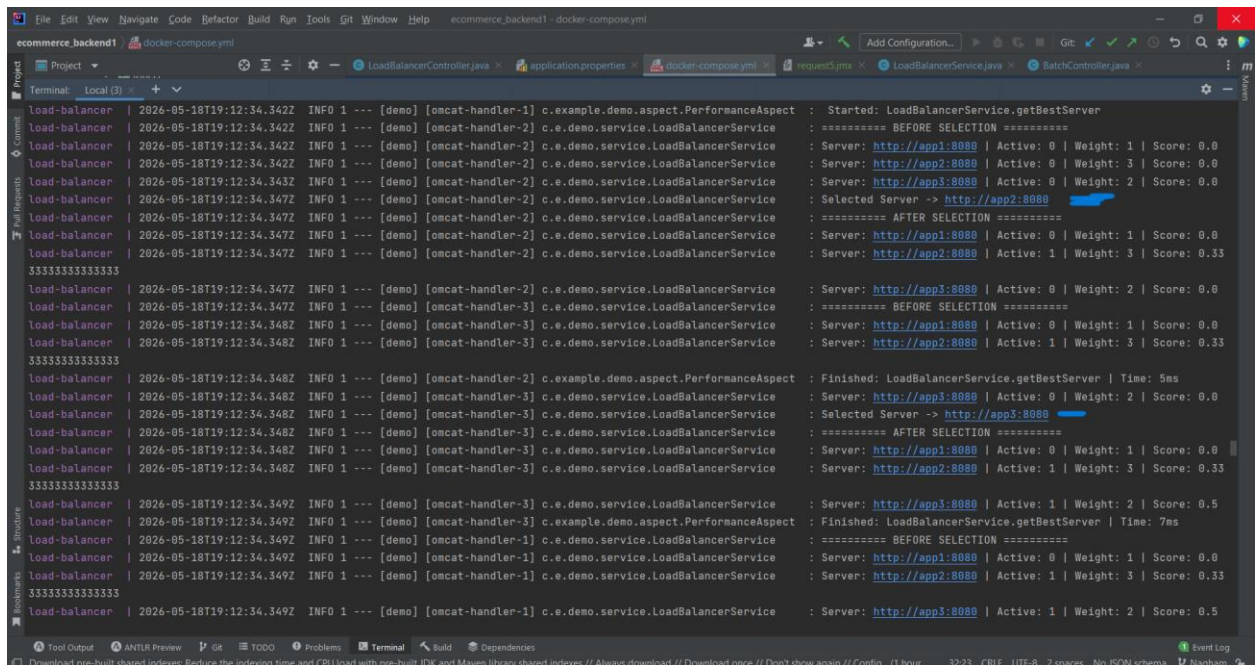
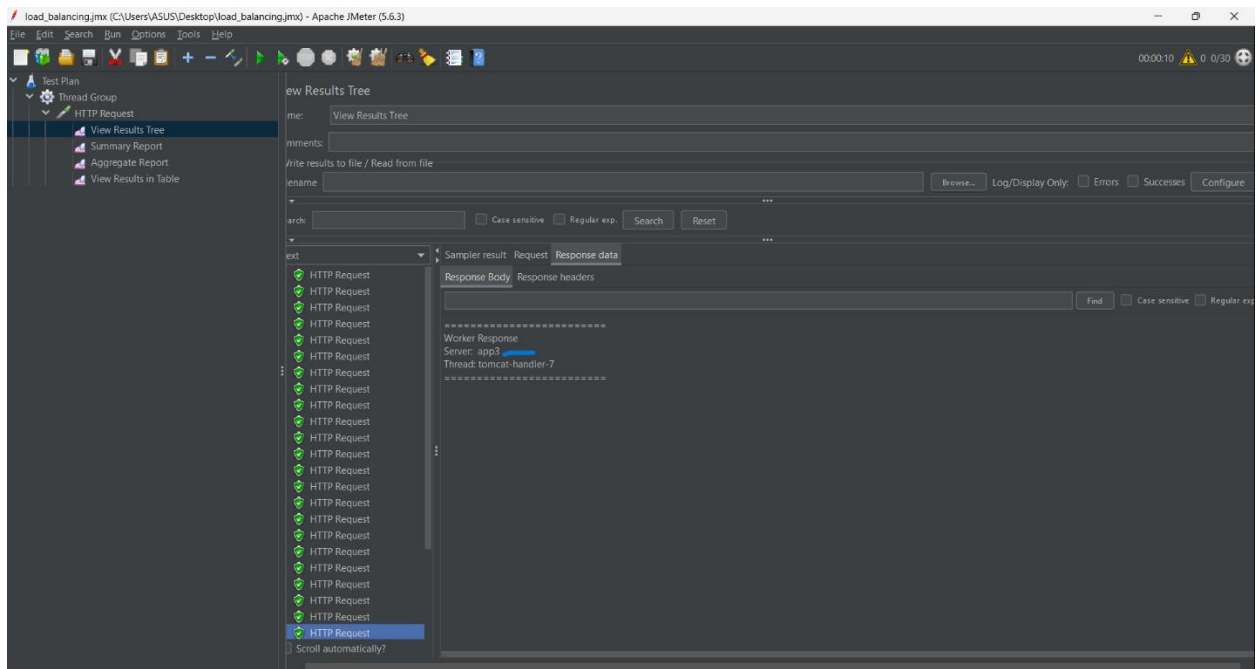
Response Body Response headers

Handled ONLY by: app1

☐ Scroll automatically?

بعد التحسين : تم توزيع الطلبات كما تحدثنا حيث تم وضع اشارات على الصور





load_balancing.jmx (C:\Users\ASUS\Desktop\load_balancing.jmx) - Apache JMeter (5.6.3)

FileEditSearchRunOptionsToolsHelp

Test Plan

Thread Group

HTTP Request

View Results Tree

Summary Report

Aggregate Report

View Results in Table

00:00:10 0 0/30

View Results in Table

Name: View Results in Table

Comments:

Write results to file / Read from file

Filename:Browse...

Log/Display Only:☐ Errors☐ SuccessesConfigure

Sample #	Start Time	Thread Name	Label	Sample Time(ms)	Status	Bytes	Sent Bytes	Latency	Connect Time(ms)
1	22:12:34.328	Thread Group 1-4	HTTP Request	10307	✓	270	132	10306	0
2	22:12:34.460	Thread Group 1-8	HTTP Request	10175	✓	270	132	10174	1
3	22:12:34.561	Thread Group 1-11	HTTP Request	10075	✓	270	132	10075	1
4	22:12:34.263	Thread Group 1-2	HTTP Request	10373	✓	270	132	10373	0
5	22:12:34.361	Thread Group 1-5	HTTP Request	10276	✓	270	132	10276	1
6	22:12:34.494	Thread Group 1-9	HTTP Request	10144	✓	270	132	10144	1
7	22:12:34.296	Thread Group 1-3	HTTP Request	10343	✓	270	132	10343	1
8	22:12:34.527	Thread Group 1-10	HTTP Request	10112	✓	270	132	10112	1
9	22:12:34.593	Thread Group 1-12	HTTP Request	10046	✓	270	132	10046	0
10	22:12:34.394	Thread Group 1-6	HTTP Request	10248	✓	270	132	10247	2
11	22:12:34.229	Thread Group 1-1	HTTP Request	10418	✓	270	132	10418	1
12	22:12:34.428	Thread Group 1-7	HTTP Request	10221	✓	270	132	10221	1
13	22:12:34.625	Thread Group 1-13	HTTP Request	10026	✓	270	132	10026	1
14	22:12:34.658	Thread Group 1-14	HTTP Request	10014	✓	270	132	10014	0
15	22:12:34.690	Thread Group 1-15	HTTP Request	10012	✓	270	132	10012	1
16	22:12:34.724	Thread Group 1-16	HTTP Request	10022	✓	270	132	10022	1
17	22:12:34.757	Thread Group 1-17	HTTP Request	10011	✓	270	132	10011	1
18	22:12:34.789	Thread Group 1-18	HTTP Request	10012	✓	270	132	10012	1
19	22:12:34.823	Thread Group 1-19	HTTP Request	10011	✓	270	132	10011	0
20	22:12:34.856	Thread Group 1-20	HTTP Request	10012	✓	270	132	10012	1
21	22:12:34.889	Thread Group 1-21	HTTP Request	10012	✓	270	132	10011	0
22	22:12:34.922	Thread Group 1-22	HTTP Request	10012	✓	271	132	10012	0
23	22:12:34.954	Thread Group 1-23	HTTP Request	10011	✓	270	132	10011	1
24	22:12:34.988	Thread Group 1-24	HTTP Request	10011	✓	271	132	10010	1
25	22:12:35.021	Thread Group 1-25	HTTP Request	10011	✓	271	132	10011	0

☐ Scroll automatically?☐ Child samples?

No of Samples: 30Latest Sample: 10011Average: 10080Deviation: 128